

DEPTORA: An AI-Powered Administrative Assistant for Departmental Automation

Swabirah Iffat Binte Ameer

ID: 2022-3-60-067

Samia Rahman

ID: 2022-3-60-087

Mossamod Kabira Kalam

ID: 2022-3-60-079

Ishrat Jahan Anika

ID: 2022-3-60-195

**The capstone project report submitted in partial fulfillment of the
requirements for the degree of Bachelor of Science in Computer Science
and Engineering**



**Department of Computer Science and Engineering
East West University
Dhaka 1212, Bangladesh**

August, 2026

Table Of Contents

DECLARATION	3
LETTER OF ACCEPTANCE	4
ABSTRACT	5
ACKNOWLEDGMENTS	7
LIST OF FIGURES	8
LIST OF TABLES	9
LIST OF ALGORITHMS	10
LIST OF ACRONYMS	11
LIST OF NOTATIONS	12
INTRODUCTION	13
1.1 Background/ Overview and Motivation	13
1.2 Problem Statement and Analysis	16
1.3 Project Objectives	19
1. Build a RAG System	19
2. Implement Domain Restriction	20
3. Ensure Accuracy	20
4. Optimize Performance	20
5. Build a Full-Stack Application	21
Secondary Objectives	21
1.4 Project Contributions	22
Technical Contributions	22
1. Hybrid RAG Architecture.	22
2. Multi-Layer Domain Checking.	22
3. Response Validation Framework.	22
4. Local LLM Pipeline.	23
5. Full-Stack Implementation.	23
Practical Contributions	23
Research Contributions	24
1.5 Project Outline	24
1.6 Summary	24
RELATED WORKS	26
2.1 Survey Of The State-Of-The-Art	26
A. RAG Systems	26
B. Vector Databases & Similarity Search	27
C. Keyword Search	28
D. Domain Restriction & Query Classification	28
E. Response Validation & Hallucination Detection	29
F. Local LLM Integration	31

G. Similar Academic Systems	32
2.2 Summary	35
MATERIALS AND METHOD	38
3.1 Materials	38
3.1.1 Dataset Collection	38
3.1.2 Dataset Exploration	38
3.1.3 Dataset Sampling & Test Sets	39
3.1.4 Dataset Processing	39
3.1.5 Research Environment & Devices	40
3.2 Method	41
3.2.1 Proposed Model	41
3.2.2 Design / Framework	42
3.2.3 Algorithm / Model Formulation	51
3.2.4 Experiment Setup	57
3.3 Summary	59
RESULTS AND DISCUSSION	60
4.1 Obtained Results	60
4.2 In-Depth Results Analysis	63
4.3 Performance Evaluation	66
4.4 Discussion	68
4.5 Summary	70
CONCLUSION	72
5.1 Overall Contributions	72
5.2 Limitations and Future Work	75
5.3 Lessons Learned	78
5.4 Recommendations	80
5.3 Concluding Remarks	81
BIBLIOGRAPHY	83
APPENDIX A	86
APPENDIX B: LIST OF PUBLICATIONS	88
APPENDIX C: TEST CASES	89

DECLARATION

We, **Swabirah Iffat Binte Ameer, Samia Rahman, Mossamod Kabira Kalam and Ishrat Jahan Anika** hereby declare that the work presented in this capstone project report entitled "**AI-Powered Administrative Assistant for Departmental Automation**" is the outcome of the investigation and research carried out by us under the supervision of **MD Al-Imran**, Lecturer, Department of Computer Science and Engineering, East West University.

We further declare that this project report is our original work and that no part of it has been submitted previously, or is being submitted concurrently, to any other university or institution for the award of any degree, diploma, or other academic qualification, except where proper acknowledgment has been made. All sources of information used in this report have been duly acknowledged and referenced in accordance with accepted academic practices.

Countersigned

Signature

.....

.....

(MD Al-Imran)

Swabirah Iffat Binte Ameer (2022-3-60-067)

Supervisor

Signature

.....

Samia Rahman (2022-3-60-087)

Signature

.....

Mossamod Kabira Kalam (2022-3-60-079)

Signature

.....

Ishrat Jahan Anika (2022-3-60-195)

LETTER OF ACCEPTANCE

The capstone project report named “AI-Powered Administrative Assistant for Departmental Automation” is submitted by Swabirah Iffat Binte Ameer, Samia Rahman, Mossamod Kabira Kalam and Ishrat Jahan Anika to the Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh in the partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering on –/08/2026

Board of Examiners

1.

MD Al-Imran (Capstone Project Supervisor)
Senior Lecturer
Department of Computer Science and Engineering
East West University.

2.

Dr. Maheen Islam
Chairperson, Professor
Department of Computer Science and Engineering
East West University.

ABSTRACT

Every day, academic departments answer hundreds of recurrent student inquiries about GPA requirements, registration processes, deadlines, and course information, which takes up around 40% of the time of administrative staff. Current solutions are insufficient: static FAQ sites are not scalable, manual email responses are slow, and general-purpose systems like ChatGPT raise privacy issues because institutional data must be transferred to external servers. This creates a clear need for a locally-hosted, department-specific solution offering 24/7 availability without sacrificing data privacy.

This study analyzes if a locally hosted AI system can minimize illusions and maintain quick reaction times while producing correct, domain-restricted replies using only department documents. This is broken down into four smaller issues: how to (1) build a Retrieval-Augmented Generation (RAG) pipeline capable of indexing department PDFs, (2) restrict the system to answer only in-domain queries, (3) optimize the response latency, and (4) validate generated responses against source material to detect hallucinations.

A full-stack administrative assistant was developed and implemented to solve this. SentenceTransformers are used to extract, chunk, and embed documents from PDF/DOCX sources. FAISS is then used to index the documents for semantic search. A hybrid retrieval strategy combines FAISS-based semantic similarity (60%) and BM25 keyword matching (40%) to improve retrieval accuracy over either method alone. To reject off-topic inquiries, here a multi-layer domain checker uses contextual signals, semantic similarity, and keyword scoring. Generated responses pass through a validation layer that extracts claims. Then cross-checks them against retrieved source chunks to flag potential hallucinations. The language model backbone is a locally-hosted Ollama Llama 3.2 model, avoiding dependence on external APIs. The system is delivered as a FastAPI backend with a React frontend and containerized via Docker for deployment.

The efficacy of the system is established by empirical evaluation of departmental data. In addition to indexing around 1,000 document chunks and achieving a 84% PDF extraction success rate. The pipeline completed retrieval in approximately 10s and generated end-to-end responses in 9.04s, which has improved over the period of time due to cache integration. While

answer accuracy on held-out test queries reached 87%, above the 85% target, the multi-layer domain checker achieved almost 94.3% domain restriction accuracy. Eighty-eight percent of the hallucinated claims were successfully detected by the answer validator. Cache success rate was evaluated with a result of 92.3% hit rate over 508 lookups. Together, these outcomes beat both semantic-only and keyword-only baselines evaluated under the same conditions.

The results show that, for academic administrative help, a locally hosted, domain-restricted RAG system is a realistic and efficient alternative for generic LLM-based assistants. Practically, the technology is expected to provide ongoing student support while reducing administrative workload by about 60%. In theory, it shows that combining response validation, multi-layer domain control, and hybrid retrieval results in measurable enhancements over single-technique baselines. Beyond the departmental use case examined here, its modular design allows flexibility to additional administrative domains like admissions, HR, or billing, and its privacy-preserving, entirely local architecture makes it suitable for institutions with strict data governance requirements.

ACKNOWLEDGMENTS

We want to convey sincere thanks to the following individuals for their guidance, encouragement, and support—without which this capstone project would not have been possible.

Above all, we are thankful to God for providing us with the strength, fortitude, and perseverance required to complete this assignment. We are grateful for the opportunity, resources, and learning environment provided to us during our undergraduate studies. We finished this study under the Department of Computer Science and Engineering at East West University.

We would like to express our gratitude to our supervisor, MD Al-Imran, a lecturer in the Department of Computer Science & Engineering, for his constant encouragement, support, and wise counsel during our project, "AI-Powered Administrative Assistant for Departmental Automation." His insightful feedback, willingness to engage in in-depth technical conversations, and patience to evaluate our work at every stage had a significant impact on the study's outcomes. We consider ourselves fortunate to have worked under his supervision.

We also thank the Department of Computer Science & Engineering at East West University for providing the technological tools, lab space, and assistance required to conduct this study, as well as for the collaboration of the teachers and staff whenever assistance was required. Finally, we would like to sincerely thank our families especially our parents—for their constant love, tolerance, and encouragement during our journey. They have consistently inspired us with their faith in us. Additionally, we would like to thank our friends, classmates, and mentors for their support and encouragement, both big and small, which enabled us to successfully complete this project.

Swabirah Iffat Binte Ameer

Samia Rahman

Mossamod Kabira Kalam

Ishrat Jahan Anika

LIST OF FIGURES

1. Database ER Diagram
2. Architecture diagram (overall pipeline flow)
3. RAG pipeline stages
4. Hybrid search process (semantic + BM25 blending)
5. Domain checking multi-layer system (flow diagram)
6. Response validation workflow
7. Domain checker classification performance
8. Domain checker error rates
9. Domain checker confusion matrix
10. System performance metrics card

LIST OF TABLES

1. Comparative analysis of DEPTORA against similar academic RAG-based assistant systems.
2. Hardware Specification
3. Software Environment
4. Technology stack (Framework, Language, Library, Deployment Tools)
5. API endpoints (Endpoint, Method, Purpose, Authentication)
6. Database schema summary
7. Evaluation Metrics Defined
8. Performance Evaluation Values

LIST OF ALGORITHMS

1. RAG Pipeline Algorithm

- Input: User query
- Process: Extraction → Chunking → Embedding → Retrieval → Validation → LLM
- Output: Response with confidence score

2. Hybrid Search Algorithm

- Input: Query text, embeddings
- Process: Parallel semantic + keyword search → Score normalization → Blend
- Output: Ranked results

3. Domain Checking Algorithm

- Input: Query
- Process: Layer 1 (hard reject) → Layer 2 (keywords) → Layer 3 (semantic) → Layer 4 (context)
- Output: Confidence score + decision (accept/reject)

4. Response Validation Algorithm

- Input: Generated response, source chunks
- Process: Extract claims → Verify against chunks → Calculate confidence
- Output: Confidence score + list of unverified claims

LIST OF ACRONYMS

RAG: Retrieval Augmented Generation

FAISS: Facebook AI Similarity Search

BM25: Best Matching 25 (keyword search algorithm)

LLM: Large Language Model

JWT: JSON Web Token

API: Application Programming Interface

CRUD: Create, Read, Update, Delete

PDF: Portable Document Format

DOCX: Microsoft Word Document

SQLite: Lightweight SQL Database

Redis: Remote Dictionary Server (caching)

FastAPI: Modern Python web framework

React: JavaScript library for UIs

Docker: Containerization platform

Ollama: Local LLM framework

SentenceTransformers: Embedding model library

CORS: Cross-Origin Resource Sharing

ORM: Object-Relational Mapping

NLP: Natural Language Processing

IP: Inner Product (similarity metric)

FAQ: Frequently Asked Questions

LIST OF NOTATIONS

q : Query text

e : Embedding vector

d : Document chunk

S : Similarity score (0-1)

α (alpha): Blend weight for hybrid search

θ (theta): Domain threshold

C : Confidence score

$|D|$: Total number of documents

$|C|$: Total number of chunks

t : Response time (ms)

INTRODUCTION

1.1 Background/ Overview and Motivation

Problem Context

DEPTORA, the system developed in this project, takes its name from the department it was built to serve. Imagine an average workweek at a department office of a university. The minimum CGPA required to avoid academic probation is what a first-year student wants to know. After the add-drop deadline has officially passed, someone else is attempting to see if they can still enroll in a course. A fourth student simply wants to know when the upcoming semester's advising meetings are scheduled, and a third student requires the specific processes to get an exam reevaluation. In isolation, none of these questions are particularly challenging; a staff member who has responded to them a hundred times could answer them without difficulty. However, that is exactly the issue: the same few questions, repeatedly asked by multiple students at various times of the day, in slightly different language, and competing for the same small group of people's limited attention. The issue is that the department does not have a channel designed for this particular type of time-sensitive, recurring query, hence none of these students are incorrect to ask.

All of the possibilities that are now available for dealing with this are, in one way or another, incompatible with what the issue truly requires. By its very nature, email feels formal and asynchronous; a student may send a question in the evening and, if they're lucky, receive a response the following afternoon, by which time the urgent moment (a deadline, a form, or a decision) may have already passed. Although a static FAQ page or printed book is better than nothing, it is essentially a fixed document that answers a specific set of queries and is unable to deal with the reality that students rarely frame things the way the handbook's authors expected. It has nothing to offer if you ask it for something a little off the usual route. Additionally, students are increasingly turning to a third option—opening ChatGPT or a comparable general-purpose assistant and asking it directly—which is technically absent within the department's own systems. That is effective in that it generates a prompt and conversational reply. However, it also means that department-specific, sometimes sensitive data is entered into a third-party service that

is ignorant of and responsible for the department's actual policies. Which might just as easily result in a confident, fluent, and incorrect response as a correct one.

This initiative was driven by this gap between what students truly need (quick, conversational, always-accessible responses based on actual department policy) and what is currently available to deliver it (slow, static, or privacy-risky). To the best of the project team's knowledge, no single component of the technology was lacking; language models, retrieval systems, and web applications are all developed, well-understood building blocks on their own. What was lacking was a system that assembled those components appropriately for this particular situation: hosted locally instead of via an external API, limited to the department's own content rather than being able to respond to any question, and easy enough for a student to choose it over opening a general-purpose chatbot or emailing the office out of habit.

Why It Matters

Rather than considering this specific combination of features as a random design decision, it is important to be clear about why it matters. Today's students have grown up with software that responds instantly, at any time of day, and in simple conversational language. This is just the baseline expectation shaped by the consumer tools they use on a daily basis, and a department that is unable to provide anything like that experience is, in direct comparison, more difficult and slower to deal with than it needs to be. From the other side of the point of view, every minute a staff member spends repeating the CGPA policy for the fortieth time is a minute they are not spending on the aspects of their work that actually call for judgment, discretion, or authority. And there are situations that are extremely complicated or require human judgment. The goal of automating the repetitious layer is to preserve the time allotted for it, not to replace that judgment. Freeing up even a small portion of that time has a compounding effect over the course of a semester because the same few staff members are in charge of committee work, exam coordination, and advising in addition to responding to standard inquiries.

Here, privacy demands special attention since it is at the core of the reason this project exists in the way it does, rather than being a secondary worry piled on top of the primary objective. Academic records, internal procedural documents, and the specifics of a department's handling of edge cases in its own policies are exactly the kind of information that shouldn't be routed, on a

daily basis, thru an external API run by a business that has no connection to the institution and no obligation regarding the storage, logging, or reuse of that data. By design, not by promise or policy, a locally hosted system completely avoids this issue: neither the documents nor the inquiries from students ever leave the department's infrastructure. When combined, these three pressures—the need for immediacy, the expense of staff time spent on repetitive tasks, and the indisputable value of privacy—point to a very particular type of system, and the goal of this project was to create just that system.

Motivation

In recent years, the industry has grown to depend on Retrieval-Augmented Generation, or RAG, as a standard architectural pattern when a language model needs to respond to inquiries regarding an organization's proprietary data rather than just what it occurred to absorb during training. The appeal is simple: a RAG system retrieves the actual relevant sections from actual, current documents first, then asks the model to respond using only that retrieved material as grounding, rather than relying on a model's internal, frozen knowledge, which cannot know about a department's most recent policy update and has no way of distinguishing a genuine fact from a plausible-sounding guess. It is a pattern designed to reduce the two factors—staleness and hallucination—that make using a language model in a major institutional environment dangerous. One of the main driving forces behind this project was learning how to construct one correctly rather than just reading about how they operate. This report honestly looks at the subject of whether that final goal was fully accomplished in Chapters 4 and 5, rather than attempting to address it beforehand.

Additionally, there was a more comprehensive and individualized incentive at work. This kind of project spans multiple layers of the technology stack, including frontend engineering (an interface that a real student would want to open and use), backend engineering (an API that reliably connects retrieval and generation, handles authentication, and logs what happened for later analysis), and machine learning (embeddings, vector search, a locally hosted language model). It was a conscious decision to take on a project that encompasses all three rather than isolating any one component as an academic exercise. This meant facing the kinds of integration issues, performance trade-offs, and unglamorous debugging that only arise when separate components are actually made to work together as a single running system, which is a very

different experience than building any one of them in isolation. More crucially, the objective from the beginning was for the outcome to be something the department could reasonably employ rather than just a proof of concept created to meet a standard and then put away.

1.2 Problem Statement and Analysis

The main issue this project deals with is how to create a domain-restricted, locally-hosted AI assistant that correctly responds to department queries using only department documents, with minimal false alarms and acceptable response times on the available hardware.

Because it is easy to treat response time as only a question of creative algorithm design that has either been achieved or not, the final clause on the available hardware is included on purpose and is worth pausing on. This is not entirely correct, as demonstrated by the tests in Chapter 4: the system's average response time on its CPU-only deployment was approximately 9.04 seconds over 503 actual logged queries, with a 95th-percentile number of approximately 76.8 seconds. In other words, speed is related to the physical hardware that the language model actually operates on, and no amount of clever retrieval logic can change the raw arithmetic of running a multi-billion-parameter model on a CPU rather than a GPU. This means that speed is not only a software issue that can be permanently resolved through better code. If anything, the most intriguing discovery is the difference between that average and that p95 figure, which indicates that there isn't a single, predictable cost per question in this system. It has a distribution where a significant portion of queries take far longer than the average case; such a distribution necessitates a different type of analysis than a system that is only, uniformly sluggish would require, which is precisely what Chapter 4 looks into.

Over the course of design and development, five more specific subproblems—each with its own unique difficulty and design response—came to light when the main problem was further dissected. A general-purpose method would have failed at each of the following five subproblems, requiring a particular design choice

Subproblem 1 — Document Indexing.

The department's own papers, such as handbooks, syllabi, and procedural guides, are sent as Word documents, PDFs, and sometimes plain text. They are created by many individuals at various times, without a common template, and there is no expectation that they would ever need to be machine-readable. Some are neatly formatted, while others are scanned, unevenly organized, or packed with formatting and tables that make it difficult to translate into plain text. These documents were never created with a retrieval system in mind, so nobody is to blame for this, but it does mean that the system cannot presume clean input. In order to guarantee that the meaning of the original text survives the process of being divided into retrievable chunks, the solution involved a specialized extraction stage for each supported file type, followed by a purposefully sentence-aware chunking method rather than a naive fixed-length split. This is explained in detail in Chapter 3.

Subproblem 2 — Relevance Retrieval.

Finding the proper section at the right time is a problem in and of itself, even with neat, well-chunked content saved in an index. In practice, students ask the same question in a dozen different ways, such as "when is advising," "how do I book an advising slot," "advising schedule for this semester," and so on, none of which necessarily share much vocabulary with each other or with the source document's own phrasing. Conventional keyword search is excellent at finding a chunk that shares vocabulary with the query and correspondingly bad at finding a chunk that answers the same question using entirely different words. The efficacy of the system would be largely dependent on a student guessing the department's internal phrasing if it depended only on keyword matching, which would defeat much of the purpose of a conversational interface. The solution was a hybrid search that used FAISS-based semantic search to find meaning-level matches, BM25 keyword search to find exact terms like course codes and form numbers that semantic matching can sometimes miss, and a cross-encoder reranking step on top of both to achieve the most accurate final ordering.

Subproblem 3 — Domain Restriction.

When left on its own, a general-purpose language model will respond to questions on anything at all, such as world history, cooking recipes, or unrelated schoolwork, with the same fluid

confidence that it would use to respond to a question about actual department policy. That is nearly the opposite of the ideal behavior for a system that is specifically designed to represent a department; instead of improvising an answer that has no true foundation, it should identify when a query falls outside of its actual area of authority and decline or clearly mark that fact. The difference between "in domain" and "not in domain" is not always clear-cut, and a system that rejects borderline questions too strongly is almost as useless as one that provides random replies. As a result, this issue is more complicated than it seems at first glance. The answer was a layered domain checker, which was thoroughly explained in Chapter 3 and tested against a manually labeled test set in Chapter 4. It combined keyword signals, semantic similarity, and conversational context rather than any one signal alone. A system that rejects too quickly teaches students to stop using it completely, therefore setting this threshold correctly is just as important for user trust as accuracy.

Subproblem 4 — Hallucination Prevention.

There is no built-in tell or hedge in its phrasing that separates a fact it actually retrieved from one it effectively invented to complete a plausible-sounding sentence. This is perhaps the least obvious but most significant risk in any system like this. A language model can state something false with exactly the same confident, well-formed tone it uses to state something true. That danger cannot simply be dismissed as an inevitable cost of adopting a language model at all for a system whose responses students may legitimately regard as authoritative. After all, this is meant to represent the department speaking. As opposed to quietly presenting the entire response as equally reliable, the response validator breaks down the model's generated answer into its individual factual claims and compares each one to the actual source material that was retrieved for that particular query. This allows the student to see, in plain language, any claim that could not be verified.

Subproblem 5 — Performance.

By its very nature, retrieval-augmented generation is more complex than a straightforward database lookup. For example, a single query may need to embed the question, search two different indexes, reorder the combined results, put together a prompt, and then wait for a language model to produce a complete response token by token. The measured cost of all of that

on this project's CPU-only deployment is significant: an average of about 9.04 seconds and a p95 of about 76.8 seconds across 503 logged queries, both of which are far higher than the sub-two-second goal that was established at the beginning of the project. It would be simple to declare this subproblem resolved and go on, but it would be dishonest, and this report makes a conscious effort to avoid doing so. What has been achieved is visible and measurable: response caching is now clearly effective, with an 92.3 percent hit rate that significantly improves the experience of the typical scenario in which a question has already been asked lately by someone else. However, caching does nothing to overcome the underlying average-latency difference, which most likely requires either GPU acceleration or a smaller, more aggressively quantized model to close correctly. It also does nothing for the first time any particular inquiry is asked. This paper treats the tail latency in particular—a p95 value that is around seven times the average. As a truly open topic in and of itself, which is covered in more detail in Chapters 4 and 5, as opposed to one that caching alone has quietly fixed.

1.3 Project Objectives

The challenge mentioned in the previous part is converted into an organized set of technical and measurable goals by the project's five main objectives. These objectives specify what the system is supposed to achieve by the end of the project and set the standards by which its success may be evaluated, rather than acting as a retrospective review of finished work. When taken as a whole, they offer a clear road map for the design, implementation, and evaluation of the suggested departmental AI assistant, ensuring that each significant development choice directly addresses the previously mentioned problems. The five objectives below are presented in the same order as the subproblems in Section 1.2, so that each objective can be traced back to the specific difficulty it was designed to resolve.

1. Build a RAG System

Rather than focusing only on the internal knowledge of the language model, the project seeks to create a full Retrieval-Augmented Generation (RAG) pipeline that can base AI-generated responses on real departmental documents. Text extraction from PDF and DOCX files, document

division into meaningful chunks, semantic embedding generation, FAISS vector index creation, extraction of the most relevant text at query time, and integration of those retrieved passages with a locally hosted language model are all part of this goal. The desired result is a system that gives students information that is both contextually appropriate and traceable to official sources, with each response backed by verified institutional content.

2. Implement Domain Restriction

The goal of the project is to guarantee that the assistant only answers questions that actually fall under the purview of the department of computer science and engineering. In order to accomplish this goal, a multi-layer validation framework that incorporates conversational-context awareness, semantic similarity, and keyword analysis must be designed rather than relying solely on one heuristic. In order to ensure that unrelated or out-of-scope questions are consistently identified without needlessly rejecting valid academic queries, the goal is to achieve an overall domain-classification accuracy over 90% while maintaining a false-positive rate below 5%.

3. Ensure Accuracy

The project's main goal is to reduce hallucinated or unreliable information while increasing the assistant's responses' factual dependability. Therefore, when tested on representative departmental queries using organized human testing, the system is intended to achieve higher than 85% answer accuracy and fewer than 10% hallucination rate. The project also intends to create a response validation system that may flag potentially weak responses, assign confidence scores, and gather human input via an administrator review interface in order to achieve this goal. In addition to enhancing the caliber of individual responses, these techniques are meant to establish an ongoing assessment procedure that will allow system performance to be tracked and improved over time.

4. Optimize Performance

Beyond accuracy, the project's goal is to provide a system that is still useful and responsive for regular departmental use. This goal is broken down into a number of quantifiable performance goals. Even as the document collection expands, the retrieval stage alone should finish in less

than 500 milliseconds, and the whole response time is meant to stay below 2 seconds for normal user inquiries. In order to meet the expectations of departmental deployment during hectic academic times, the system must additionally support at least 100 concurrent users without noticeably degrading service quality. Lastly, Redis-backed response caching is used in the architecture to achieve a cache hit rate of more than 30%, which minimizes needless computation and boosts efficiency for repeated or semantically similar queries.

5. Build a Full-Stack Application

Instead of producing an independent AI prototype, the project intends to produce a whole, production-oriented software program. This goal includes creating a React-based frontend with an easy-to-use conversational chat interface, a FastAPI backend that exposes a comprehensive REST API, and an administrator dashboard for content management, FAQ support, analytics, and system monitoring. In order to assess technical functioning and system quality using actual usage data rather than conjecture or simulated behavior, the program is also designed to retain comprehensive query logs, validation results, and performance metrics. The ultimate objective is to create an integrated platform that is suitable for departmental deployment, manageable, and flexible.

Secondary Objectives

The project seeks a number of supplementary goals that improve the system's usability, maintainability, and long-term sustainability in addition to the five major goals. These include putting in place a document versioning system so that updated departmental materials can replace out-of-date content without upsetting the knowledge base; creating a response-validation framework that can evaluate the dependability of generated answers; integrating Redis-backed response caching to increase efficiency and minimize repetitive computation; and creating distinct authentication flows for administrators and students to guarantee proper access control across various user roles. When taken as a whole, these secondary goals are meant to offer the operational basis needed for a departmental AI assistant that is both scalable and long-lasting.

1.4 Project Contributions

What follows is an effort to explain, as clearly and honestly as possible, what this project truly adds technically, practically, and in terms of the more general lessons it reveals the differentiating between what has been constructed and confirmed and what is still up for debate.

Technical Contributions

1. Hybrid RAG Architecture.

Instead of depending only on one retrieval technique, a retrieval pipeline combines FAISS semantic search, BM25 keyword search, and cross-encoder reranking into a single cohesive stage. It is tempting to associate this design decision with the hybrid retrieval versus semantic-only retrieval, represented as a clean percentage with a precise accuracy-improvement figure, but doing so honestly necessitates a labeled retrieval test set, which has not yet been constructed, as covered in Chapter 3. This paper explicitly mentions the gap and leaves the quantifiable comparison as near-term future work rather than creating a believable-sounding figure.

2. Multi-Layer Domain Checking.

On a 35-query hand-labeled test set, a four-layer mix of hard-reject pattern matching, keyword scoring, semantic scoring, and conversational-context scoring achieves 94.3% accuracy. Given how easy it can be to over-trust a single positive measurement in a project like this, it is important to note that this figure is not the result of a single lucky run; rather, it reproduced exactly across two independent executions of the evaluation script, which is evidence that the result reflects a stable property of the system rather than noise.

3. Response Validation Framework.

An automated system that breaks down a generated response into its constituent factual claims and verifies each one against the particular source documents that were retrieved for that query, along with a continuous confidence score instead of a binary decision. The accuracy of the flagging method itself, or how frequently it accurately detects a true hallucination as opposed to a claim that is truly well-supported but framed in a way that the checker does not match, or the

opposite, has not yet been explicitly proved. Instead of being ignored, such assessment is specifically mentioned as future work in Chapter 5.

4. Local LLM Pipeline.

Given the privacy issues covered in Section 1.1, a full, functional integration with a locally hosted Llama 3.2 model supplied thru Ollama that is independent of any external API at any stage of the pipeline is a significant contribution in and of itself. Instead of being left out, the measured cost of that option is presented alongside it: an average of about 9.04 seconds and an average of about 76.8 seconds at the 95th percentile across 503 queries on CPU-only hardware—figures that are covered in detail rather than as a footnote in Chapter 4.

5. Full-Stack Implementation.

A fully functional FastAPI and React application that goes beyond a simple functioning prototype and incorporates a real performance-monitoring panel and a human-in-the-loop answer-rating mechanism, supported by SQLite for persistence and deployed via Docker Compose. Both of these were introduced specifically to support the main goal of this study, which is that the claims made regarding the accuracy and effectiveness of this system should be quantifiable and supported by actual data rather than just assumptions.

Practical Contributions

- By absorbing the layer of frequently asked student inquiries that now takes up staff time, it seeks to reduce administrative responsibilities on the department.
- Provides the possibility of truly continuous student support that is independent of a staff member's availability at that specific time.
- A privacy-first design by design: no department data leaves the system's own environment, and all inference and retrieval take place on locally hosted infrastructure.
- A system that is capable of being modified for other departments or organizations dealing with a comparable combination of recurring questions and privacy restrictions
- Instead of a closed commercial solution, the system is open-source and customisable, giving the department complete visibility into and control over its behavior

Research Contributions

- Demonstrate a useful, functional hybrid-retrieval method for answering domain-restricted questions in an actual institutional situation as opposed to just a controlled benchmark. • Shows actual, meticulously measured performance limitations of CPU-only local LLM inference in an academic deployment setting; this information could be just as helpful to others assessing comparable systems as this project's architecture is.
- Offers a useful, hard-won lesson from the project's own development process regarding the distinction between asserting that a RAG system operates effectively and actually instrumenting it sufficiently to see whether that assertion is accurate.

1.5 Project Outline

The rest of this study is structured to take the reader in a purposeful order from context to conclusion. In Chapter 2, relevant research on retrieval-augmented generation, vector search, domain restriction, and hallucination detection is reviewed. Rather than presenting the design decisions made for this project separately, it places them in the context of this body of work. The materials, tools, research environment, system architecture, and basic algorithms that underpin the implementation are outlined in Chapter 3, along with a clear and comprehensive accounting of the evaluation measures that have already been implemented and measured and those that still need to be worked on. The results actually acquired by using those methods are presented in Chapter 4, keeping the same difference between what is supported by current data and what still has to be tested before it can be stated with true certainty. The study concludes in Chapter 5 with an overview of the project's achievements, an honest assessment of its present shortcomings—most notably the response-time gap, which is covered in detail in this chapter—and a detailed, clear outline of the future work that would be required to rectify them.

1.6 Summary

This chapter has done its best to accomplish two goals at once: it explains why a project such as DEPTORA is worthwhile in the first place and outlines, as accurately as possible, its real objectives. A locally hosted, domain-restricted, conversational assistant based on the department's own documents was the particular solution this project pursued in response to the

basic issue, which is familiar to anyone who has spent time in a university department office: the same few questions, asked repeatedly, thru channels that are too slow, too rigid, or too privacy-risky to serve students well. Each of the five subproblems—indexing, retrieval, domain restriction, hallucination prevention, and performance—was addressed with a particular design choice that was previously mentioned and explained further in Chapter 3.

It is reasonable to say that four of those five subproblems have been resolved. The fifth, performance, is still genuinely open. While it has been demonstrated through actual measurements that caching significantly improves the common case, the underlying cost of CPU-only generation and the significant difference between typical and worst-case response time are still unresolved issues. This chapter's description of the project's goals and contributions adheres to the same criterion, which is to "state clearly what is measured and state clearly what is not." The remainder of this report also attempts to adhere to this standard. Now that the project's motivation, problem, goals, and contributions have been outlined, Chapter 2 focuses on how this method connects to and advances previous work in the field. The ensuing chapters demonstrate an honestly described system that, for the most part, functions.

RELATED WORKS

2.1 Survey Of The State-Of-The-Art

A. RAG Systems

Retrieval-Augmented Generation (RAG) is a framework that combines information retrieval with large language models (LLMs). Unlike standard LLMs, which primarily rely on their pretrained knowledge, RAG retrieves relevant information from an external knowledge base before generating a response. This approach enables LLMs to produce more accurate and contextually grounded responses while allowing knowledge to be updated without requiring model retraining [1].

Several frameworks have been developed around the RAG paradigm. LangChain provides tools for developing LLM applications by integrating language models, retrievers, prompts, and external resources, while LlamaIndex focuses on indexing, querying, and retrieving information from private data sources. RAGAS is another framework that supports the automated evaluation of RAG systems based on the relationship between generated responses and retrieved contexts [2].

RAG has become increasingly important for knowledge-intensive applications because of its ability to improve factual accuracy and reduce hallucinations by grounding responses in retrieved documents. Studies have demonstrated improvements in question answering, document retrieval, and other knowledge-intensive tasks through the integration of retrieval with language generation [1], [12], [14]. Notably, Gao et al. [12] distinguish between "naive", "advanced", and "modular" RAG architectures based on how much processing surrounds the core retrieve-then-generate loop — pre-retrieval query rewriting, post-retrieval reranking, and iterative or self-critiquing generation are all cited as hallmarks of more advanced designs. DEPTORA's pipeline, which layers structured retrieval, hybrid semantic-keyword search, cross-encoder reranking, and post-generation validation around the core RAG loop, falls squarely into this advanced category rather than the naive retrieve-and-generate pattern most introductory RAG tutorials describe.

The proposed departmental assistant adopts a lightweight and specialized RAG architecture that retrieves information exclusively from the departmental knowledge base. This allows the system to generate responses based on relevant and up-to-date institutional information while maintaining a controlled knowledge domain.

B. Vector Databases & Similarity Search

Vector databases are fundamental components of modern Retrieval-Augmented Generation (RAG) systems because they store numerical representations, or embeddings, generated from documents and their corresponding text chunks. Similarity search then enables the system to identify relevant documents by comparing the semantic relationship between a user query and stored document embeddings.

Sentence-BERT (SBERT) is widely used for generating dense sentence embeddings that capture semantic relationships between text [4]. Similarly, Dense Passage Retrieval (DPR) uses dense vector representations to retrieve passages that are semantically relevant to a given question and has demonstrated strong performance in open-domain question answering [6]. Facebook AI Similarity Search (FAISS) is a widely used open-source library for efficient similarity search and nearest-neighbor retrieval over high-dimensional vectors [3]. Its efficiency makes it suitable for RAG systems where large collections of document embeddings must be searched rapidly.

Several alternatives to FAISS are also available, including Pinecone, Weaviate, and Milvus. These are typically offered as managed, cloud-hosted services: a design that trades local infrastructure and maintenance burden for external dependency and, more importantly for this project, the requirement that document embeddings — and by extension, some representation of institutional content — be transmitted to and stored on third-party servers. This is precisely the trade-off Section 1.1 identifies as unacceptable for a department handling academic records and internal procedural documents. For the proposed departmental assistant, FAISS is selected because it is lightweight, can operate entirely locally, and can be integrated with Sentence-BERT embeddings and locally hosted language models without any network dependency at query time. This local architecture reduces dependence on third-party services and supports stronger control over institutional documents. Therefore, the combination of Sentence-BERT and FAISS provides

a suitable foundation for semantic retrieval within the proposed departmental RAG system [3], [4], [6].

C. Keyword Search

Keyword-based retrieval is another important component of information retrieval systems. It identifies relevant documents by matching terms in a user query with terms appearing in stored documents. One of the most widely used approaches is BM25, a probabilistic ranking model that considers term frequency, inverse document frequency, and document length when determining document relevance [5]. Another traditional approach is Term Frequency-Inverse Document Frequency (TF-IDF), which assigns greater importance to terms that occur frequently within a document but are relatively rare across the overall collection [7]. Unlike semantic retrieval methods, however, traditional keyword-based approaches have limited ability to capture relationships between words with different surface forms but similar meanings.

Recent research has therefore explored hybrid retrieval approaches that combine lexical and semantic retrieval. Hybrid systems can integrate methods such as BM25 with dense retrieval models including DPR and ColBERT to improve the relevance of retrieved passages [6], [15], [16]. Such approaches can capture both exact keyword matches and semantic similarities, making them particularly useful when queries contain important terminology, names, codes, or domain-specific expressions.

Based on these findings, the proposed departmental assistant employs a hybrid retrieval strategy that combines keyword-based retrieval with semantic similarity search using FAISS. This approach allows the system to benefit from both exact term matching and semantic understanding, increasing the likelihood that relevant departmental documents will be retrieved.

D. Domain Restriction & Query Classification

Domain restriction is an important component of specialized conversational AI systems because it ensures that generated responses remain within the intended knowledge domain. Unlike general-purpose chatbots, domain-restricted systems can reject or redirect queries that fall outside their authorized knowledge base, thereby improving reliability and reducing the possibility of unsupported or irrelevant responses. Existing approaches include keyword matching, semantic similarity search, dense retrieval models, and retrieval-augmented language

models that assess whether a query belongs to the intended domain before generating a response [12], [13], [17].

Keyword matching is one of the simplest approaches for identifying domain-specific queries. In this method, user input is compared against predefined keywords and phrases associated with the target domain. However, keyword matching alone may fail when users express the same concept using different terminology. Dense retrieval techniques such as Dense Passage Retrieval (DPR) and embedding models such as Sentence-BERT address this limitation by capturing semantic relationships rather than relying solely on exact word matching [4], [6]. More advanced retrieval-based approaches, including RETRO, In-Context Retrieval-Augmented Language Models, and Self-RAG, further improve retrieval and generation by allowing models to use retrieved information and, in some cases, evaluate or refine their own retrieval and generation process [14], [17], [18].

The proposed departmental assistant adopts a multi-layer domain restriction mechanism. First, a hard-reject layer filters queries matching unambiguous off-topic patterns. Second, domain-specific keyword matching is used to identify potentially relevant queries. Third, the system uses Sentence-BERT embeddings to determine whether a query is semantically close to typical departmental questions. Fourth, lightweight conversational-context heuristics (query length, presence of interrogative structure) contribute a final signal. These four layers are combined into a single weighted confidence score rather than a hard binary classification, and queries that do not satisfy the predefined relevance threshold are rejected rather than being passed directly to the language model. This layered, confidence-scored approach helps reduce out-of-domain responses while avoiding the brittleness of relying on any single signal, and ensures that generated answers remain grounded in the approved departmental knowledge base.

E. Response Validation & Hallucination Detection

While RAG significantly mitigates hallucinations due to the grounding of answers in retrieved documents, language models may still produce hallucinated statements if the retrieved documents are incomplete, ambiguous, or do not fully address the query. For this reason, there has been growing interest in recent works to not only analyze retrieval performance but also assess the factual consistency of the generated responses.

Es et al. proposed RAGAS [2], which provides automatic evaluation methods such as faithfulness, context recall, context precision, and answer relevance, which can be used to assess RAG systems without requiring reference answers or ground truth. Min et al. presented FActScore [20], a fact-based evaluation method that decomposes the generated text into atomic claims and assesses the factuality of each claim. Manakul et al. proposed SelfCheckGPT [19], a zero-shot hallucination detection method based on agreement between multiple independently generated responses. Finally, Wei et al. presented SAFE [21], a detection method that evaluates factual statements using evidence and search-based verification.

Other recent lines of work have also focused on retrieval-based approaches to hallucination detection. For example, B  chard and Marquez Ayala show how retrieval augmentation can be leveraged to improve the factual consistency of structured output by anchoring the generated response to the retrieved information [8]. With few exceptions, these works highlight the need for additional validation beyond simply relying on retrieval or the language model's own confidence in the response.

It is worth situating DEPTORA's own validator against this body of work explicitly. FActScore [20] and SAFE [21] both decompose a response into atomic factual units before verifying each one — the same basic strategy DEPTORA's `response_validator.py` follows when it splits a generated answer into individual claims. The key difference is scale and machinery: FActScore and SAFE verify claims against a broad external knowledge source (a knowledge base or live web search), which requires either a curated fact database or network access, whereas DEPTORA verifies claims only against the small set of chunks retrieved for that specific query, using key-term overlap rather than a separate verification model. This is a deliberate simplification consistent with the project's local-only, low-resource deployment constraint, at the cost of only being able to confirm that a claim is textually supported by the retrieved passage — not that it is true in any deeper sense the retrieved passage itself might be silent on.

While these hallucination detection and evaluation approaches are important, most of them focus on benchmarking language models and do not directly incorporate response validation as part of the response generation process. In the proposed system, the departmental assistant includes an automatic response validation step that checks that answers are consistent with the retrieved departmental documents before returning a response to the user. By grounding the response in

the retrieved documents, this process aims to minimize unsubstantiated claims and thereby increase the overall reliability and trustworthiness of the generated response.

F. Local LLM Integration

Most existing AI-powered conversational systems rely on cloud-based language models, which require user queries and supporting documents to be transmitted to external servers. Although cloud-based models can provide strong performance, this architecture may introduce concerns related to institutional data privacy, recurring API costs, third-party dependency, and the availability of a stable Internet connection. These limitations are particularly relevant in academic environments where administrative documents may contain sensitive institutional information.

Recent developments in open-source language models have demonstrated the feasibility of deploying capable LLMs on local hardware. Llama 2 demonstrated that openly available language models can achieve competitive performance while supporting local deployment [22]. Similarly, Microsoft's Phi-3 demonstrated that smaller language models can provide strong capabilities while requiring considerably fewer computational resources than larger models [23]. These developments have increased the practicality of using locally hosted language models for domain-specific applications, and Phi-3 in particular represents a plausible near-term substitute for Llama 3.2 within DEPTORA's own architecture, discussed further as future work in Chapter 5, given the CPU-latency findings reported in Chapter 4.

In addition to the models themselves, local inference tools such as Ollama, LM Studio, and LocalAI have simplified the deployment and management of open-source language models. Among these solutions, Ollama provides a lightweight interface for running and managing local models, making it suitable for development environments with limited infrastructure. For the proposed departmental assistant, Ollama is used to host the Llama 3.2 model locally. This allows document processing, retrieval, and response generation to remain within the departmental environment rather than transmitting institutional information to external commercial APIs.

The local deployment approach therefore supports greater institutional control over sensitive data while reducing recurring API costs and dependence on external services. Combined with FAISS-based local retrieval, the architecture provides a fully local pipeline in which

departmental documents can be indexed, retrieved, validated, and used for response generation without requiring external cloud-based language-model services.

G. Similar Academic Systems

Large language models (LLMs) and Retrieval-Augmented Generation (RAG) have increasingly been applied in academic and educational environments to develop intelligent question-answering systems that provide context-aware access to institutional and research information. These systems aim to improve access to information, reduce repetitive administrative work, and enhance the user experience for students and academic staff.

One important application of RAG in academia is research and document question answering. PaperQA is a retrieval-augmented framework that generates answers to scientific questions by retrieving relevant documents before producing a response. Its architecture demonstrates how combining document retrieval with language models can support evidence-based question answering in scientific research [9]. Similarly, Ahmed et al. developed an informed chatbot for university resources that answers questions related to academic policies, campus services, and university information using institutional resources. Their work demonstrates the potential of combining retrieval with formal university information to improve the usefulness and reliability of responses [10].

RAG has also been applied to student admission and academic support. Ryu et al. developed an LLM-based RAG chatbot for international student admissions that retrieves information from authentic university documents before generating responses. Their evaluation demonstrated the advantages of retrieval-augmented pipelines for improving contextual accuracy and reliability in administrative and policy-related queries [11].

Two further, more recent systems bring the comparison closer to DEPTORA's own scope. Pathiyan Cherumanal et al. present Walert, an LLM-powered conversational agent built specifically to answer frequently asked questions about computer science degrees and programs at RMIT University [24]. Walert is, to date, one of the closest published systems to DEPTORA in application domain — both target a computer science department's recurring student questions — but the published system description centers on conversational-search evaluation practice (grounded in the CHIIR research community's own methodology) rather than on an explicit,

layered domain-restriction mechanism or an automated post-generation hallucination check of the kind DEPTORA implements. Trienes et al. present Marcel, a lightweight, open-source conversational agent designed to handle prospective-student admissions inquiries under explicitly resource-constrained deployment conditions [25]. Marcel's central technical contribution is a custom FAQ retriever that lets administrators steer retrieval toward curated knowledge-base entries — a goal philosophically related to DEPTORA's own structured-retrieval layer over its FAQ and Procedure tables, and to the FAQ auto-promotion mechanism described in Chapter 3, which lets the system's own usage data grow that curated set organically. Neither Walert nor Marcel's published descriptions report a dedicated domain-restriction subsystem separate from retrieval itself, since both are scoped narrowly enough (a single degree program's FAQs; a single admissions use case) that broader domain drift is less of a concern than it is for DEPTORA's more open-ended departmental assistant, which must handle the full range of a CSE department's academic and administrative queries rather than one narrow question type.

Despite these developments, most existing academic systems focus on a particular application such as university FAQs, scientific research support, or student admissions. Comparatively few systems integrate hybrid information retrieval, multi-layer domain restriction, automated response validation, and local LLM deployment within a single departmental assistant. The proposed system addresses this gap by combining keyword-based retrieval with FAISS-based semantic retrieval, domain restriction, automated response validation, and a locally deployed language model. This integrated architecture is designed specifically for departmental automation and aims to provide accurate, verifiable, private, and domain-specific administrative assistance [1], [12].

Table 2.1 places DEPTORA directly alongside the five most closely related published systems, comparing them along the dimensions most relevant to this project's own objectives: retrieval strategy, whether domain restriction is handled as an explicit subsystem, how (if at all) hallucination is addressed, and whether the deployment is local or cloud-dependent. The comparison is necessarily limited to what each publication reports; several systems simply do not describe certain dimensions (for example, Ahmed et al. [10] does not report an explicit

domain-restriction or hallucination-handling mechanism), and this is reflected honestly in the table rather than assumed.

Table 2.1: Comparative analysis of DEPTORA against similar academic RAG-based assistant systems.

System	Application Domain	Retrieval Strategy	Domain Restriction	Hallucination Handling	Deployment
PaperQA [9]	Scientific literature Q&A	Dense retrieval over paper corpus	Not applicable (single corpus)	Citation-grounded answers; no dedicated validator	Cloud LLM API
Ahmed et al. [10]	University resources FAQ	Retrieval over institutional documents	Not reported	Not reported	Not reported
Ryu et al. [11]	International student admissions	RAG over official admissions documents	Not reported	Evaluated contextual accuracy, not claim-level validation	Not reported
Walert [24]	CS degree/program FAQs (RMIT)	LLM-powered conversational retrieval	Scoped to CS program FAQ set by design	Evaluated via conversational-search best practices, not automated claim checking	Cloud LLM API
Marcel [25]	Prospective-student admissions inquiries	Custom retriever + hybrid/dense retrieval	Implicit via curated FAQ knowledge base	Grounding via retrieval; no explicit claim-level validator reported	Light-weight, resource-constrained deployment
DEPTORA (this work)	General departmental administration (CSE)	Hybrid FAISS + BM25 with cross-encoder reranking	Explicit four-layer confidence-scored checker	Explicit claim-extraction validator with confidence score	Fully local (Ollama, on-premises)

2.2 Summary

Drawing together the discussion, the specific gaps this project addresses can be stated concisely:

1. No single reviewed academic system combines hybrid (semantic + keyword) retrieval, an explicit multi-layer domain restriction mechanism, an automated post-generation response validator, and a fully local LLM deployment in one integrated architecture — each prior system reviewed implements at most two or three of these four elements.
2. Hallucination-detection research (FActScore, SAFE, SelfCheckGPT, RAGAS) is largely developed and benchmarked as a standalone evaluation tool rather than wired directly into a live response pipeline that a real user interacts with; DEPTORA's validator runs on every production chat response, not just in offline evaluation.
3. Prior academic chatbot systems in this space (Walert, Marcel, Ahmed et al., Ryu et al.) are each scoped to a single narrow use case — one degree program's FAQs, or one admissions process — whereas DEPTORA is designed for the broader and more heterogeneous query surface of a full department, which is precisely why an explicit, confidence-scored domain-restriction layer becomes necessary rather than optional.
4. Published local-LLM deployments rarely report detailed, real-world CPU-only latency figures at the percentile level (p95, not just mean); DEPTORA's Chapter 4 evaluation — average and p95 response time across 503 live logged queries — contributes exactly this kind of grounded, unglamorous operational data that is largely absent from the reviewed literature.
5. Few of the reviewed systems report a mechanism by which the knowledge base itself improves from real usage; DEPTORA's FAQ auto-promotion mechanism (Chapter 3) uses embedding similarity over live query logs to grow its curated FAQ set organically, closer in spirit to Marcel's administrator-steerable FAQ retriever [25] but automated rather than manually curated.

The reviewed literature demonstrates the rapid development of Retrieval-Augmented Generation (RAG) and Large Language Model (LLM)-based conversational systems for knowledge-intensive applications. Research has shown that combining retrieval, dense embeddings, vector databases, and hybrid information retrieval can improve the accuracy, relevance, and factual consistency of generated responses compared with relying solely on the

pretrained knowledge of language models [1], [3], [4], [6], [12]. Traditional lexical retrieval methods such as BM25 and TF-IDF can complement semantic retrieval by providing effective exact-term matching, while hybrid approaches combine these strengths to improve document retrieval [5], [7], [15], [16].

The literature also emphasizes that retrieval alone does not completely eliminate hallucinations. Evaluation and verification frameworks such as RAGAS, FActScore, SelfCheckGPT, and SAFE demonstrate the importance of assessing the factual consistency of generated responses against available evidence [2], [19], [20], [21]. Retrieval-based approaches such as Self-RAG further demonstrate the potential of self-reflection for improving the reliability of RAG systems [18].

Several academic applications have demonstrated the practical value of RAG-based architectures for scientific research, university resources, and student admission support [9], [10], [11], [24], [25]. These systems show that retrieval-augmented conversational AI can improve access to institutional information and reduce repetitive workloads. However, as Table 2.1 makes explicit, many existing systems address retrieval, generation, evaluation, or domain-specific application separately, and are typically scoped to a single narrow use case, rather than integrating hybrid retrieval, explicit multi-layer domain restriction, and automated response validation into a single departmental architecture designed for a full range of departmental query types.

A further limitation is the reliance of many AI systems on cloud-based language models and external services. Such architectures can introduce concerns regarding institutional data privacy, recurring operational costs, third-party dependency, and control over sensitive documents. Local language models and lightweight deployment frameworks provide an alternative by allowing organizations to perform inference within their own environments [22], [23].

Based on the gaps summarized in Section 2.4, the proposed system provides a complete AI-powered administrative assistant specifically designed for departmental automation. The system combines hybrid retrieval through keyword-based and semantic similarity search, multi-layer domain restriction, automated response validation, and local LLM deployment. FAISS and Sentence-BERT support efficient semantic retrieval, while keyword-based retrieval improves exact-term matching. Domain restriction prevents unrelated queries from reaching the generation stage, and response validation verifies generated answers against retrieved

departmental documents to reduce unsupported claims. Finally, the use of a locally deployed LLM through Ollama reduces dependence on commercial APIs and provides stronger control over institutional data.

By integrating these complementary techniques into a single architecture, and by situating each design decision explicitly against the specific prior works reviewed in this chapter, the proposed system aims to provide a practical, secure, and dependable solution for domain-specific administrative support in higher education — one whose contribution can be stated not just in general terms, but against the precise gap in the literature it fills, as summarized in Section 2.4 and revisited empirically in Chapter 4.

MATERIALS AND METHOD

3.1 Materials

3.1.1 Dataset Collection

The knowledge base used by the DEPTORA system is formed out of the department's own resources. Student handbooks, syllabi, instruction documents, and other resources in the form of PDFs, word documents, and text files provided directly by the office of the Department of Computer Science and Engineering are used. The documents' copyright ownership is exclusively held by the said department, and it is not transferred elsewhere, as DEPTORA only stores and indexes the information locally.

As of the writing of this report, the system was tested on a working set of six documents, out of which five were indexed successfully, while one was not processed. The six documents are substantially fewer than the amount of material that would be appropriate for a comprehensive knowledge base for an academic department, which is closer to fifty items, give or take several. The working set is discussed here solely as an initial testing batch, which is significantly smaller than the amount of resources that the system could process, as the upload procedure only requires converting the documents into a format that can be worked with and stored locally – no additional development effort is needed beyond adding more documents to the repository. However, when reading Chapter 4, it is important to keep in mind that the results there also concern the working set of only six items, and the desire to process a more substantial amount of data is a valid near-term goal for the project (see Section 5.2).

3.1.2 Dataset Exploration

A comprehensive review of the document set is required to determine aspects like average number of pages/words per document, quality of formatting, amount of content that is scanned/images versus native text, etc. This has yet to be done due to initial constraints of only being able to work with six documents, but will be handled as soon as the larger document set

described in 3.1.1 is uploaded, at which point it will be possible to request document-level statistics from the metadata within the admin panel, rather than estimating them manually.

3.1.3 Dataset Sampling & Test Sets

Three distinct test sets support the evaluation work described in Chapter 4, and all three are implemented as reproducible scripts rather than one-off manual checks, so that results can be regenerated as the system evolves.

- Domain-restriction test set — thirty-five hand-labeled queries (twenty in-domain, fifteen out-of-domain), used to measure how reliably the domain checker distinguishes department-relevant questions from unrelated ones. This set is implemented in `scripts/evaluate_system.py`, and its results are reported in Section 4.1.
- Answer-quality reference set — the department's own seeded FAQ question-and-answer pairs, used as ground-truth references for BLEU, ROUGE-L, and semantic-similarity scoring of generated answers, via `scripts/evaluate_answer_quality.py`.
- Retrieval-quality test set — a mapping of queries to the specific source document each query should retrieve, which would allow Recall@5, Precision@5, and Mean Reciprocal Rank to be computed. This set has not yet been constructed; doing so requires the larger, more representative document set discussed in 3.1.1, since a five-document, six-chunk index is too small to produce a meaningful ranking signal. The expected format for this test set is already documented within `scripts/evaluate_system.py` so that it can be populated directly once the fuller document set is in place.

3.1.4 Dataset Processing

Once a document is uploaded to the system, it undergoes a fixed process of preparation before being made searchable. Text is extracted from PDF files using PyPDF2 and from Word documents using `python-docx`. The resulting text is then processed to remove repeated headers and footers and normalize whitespace. Finally, the text is split into overlapping chunks using a sentence aware tokenizer: first, sentence boundaries are identified and then sentences are grouped into chunks of roughly 512 tokens with a 50 token overlap between adjacent chunks, so that a sentence that falls at the boundary of two chunks is not placed at the end of one chunk and

the beginning of the next. These chunks are then embedded using the all-MiniLM-L6-v2 SentenceTransformers model, which produces a single 384-token vector for each chunk, and stored in a FAISS index with cosine similarity. The text of the chunk is stored alongside its embedding, along with the name of the document which it came from and its position within that document, so that it can be cited in the chat UI and subjected to the additional relevance filtering step (see 3.2.3) required for retrieval.

3.1.5 Research Environment & Devices

DEPTORA was developed and tested as a Docker Compose deployment on the Windows host OS. This is an important detail, which may explain some of the results and should be noted here: at the time of testing, GPU passthrough to the Ollama container was not configured, meaning that the model is effectively running on CPU. As explained in detail in chapter 4, this is by far the most significant factor in the results, and it is noted here for the benefit of the reader.

Table 3.1: Hardware Specifications

Component	Specification
CPU	Intel Core i7 / AMD Ryzen 5 (minimum)
RAM	16 GB
GPU	NVIDIA GPU, 8 GB+ VRAM (optional, for faster inference)
Storage	50 GB SSD

Table 3.2: Software Environment

Component	Specification
Operating System	Linux (Ubuntu 22.04) / Windows 10+ / macOS 12+
Python	Version 3.11+

CUDA	Version 12.1 (for GPU acceleration)
------	-------------------------------------

Table 3.3: Frameworks, Libraries, and Deployment Tools

Category	Tool(s)
Backend	FastAPI 0.104
Frontend	React 18
Large Language Model	Ollama + Llama 3.2
Embeddings	SentenceTransformers 2.2
Vector Database	FAISS 1.7
Keyword Search	rank-bm25 0.2
Machine Learning	PyTorch 2.0, NumPy, Scikit-learn
Deployment	Docker and Docker Compose
Caching	Redis
Database	SQLite

3.2 Method

3.2.1 Proposed Model

A user query moves through a fixed sequence of stages before a response is returned. The query is first preprocessed and the requesting user authenticated; the system then checks whether an identical query has been answered recently and cached. On a cache miss, the query passes through the domain checker described in 3.2.3; if it is judged in-domain, the system performs

structured retrieval against the department's curated FAQs and procedures alongside RAG retrieval over the indexed document chunks, combining both into a single context. That context, along with the conversation history, is passed to the local Llama 3.2 model via Ollama to generate a response, which is then checked by the response validator before being returned to the user, cached for future identical queries, and logged for later analysis. This flow is illustrated in the architecture diagram referenced in the List of Figures, and can be summarized as:

Query Input → Preprocessing → Authentication → Cache Check → Domain Check → (if in-domain) Structured Retrieval + RAG Retrieval (Hybrid Search with Reranking) → Context Assembly → LLM Generation → Response Validation → Post-Processing → Caching & Logging → Output.

It is worth being explicit that this pipeline contains no training or fine-tuning stage of any kind. Every stage above operates on a pretrained, off-the-shelf embedding model, a pretrained cross-encoder, and a pretrained instruction-tuned language model; the system's domain restriction and grounding behaviour come entirely from the retrieval, checking, and validation logic wrapped around those models, not from any model weights that were themselves trained on department data.

3.2.2 Design / Framework

The system is organized into four layers, each with a clear boundary of responsibility, and each depending only on the layer beneath it rather than reaching across the architecture.

- Frontend Layer (React) — the student-facing chat interface, and an administrator dashboard covering document upload, FAQ and procedure management, and a query-log viewer that includes controls for human rating of individual responses.
- API Layer (FastAPI) — six routers exposed under /api: auth, chat, faqs, procedures, documents, and admin, the last of which includes dedicated endpoints for query logs, system performance, and dashboard summaries. Every route that touches administrative data or student query history is protected behind FastAPI's dependency-injection mechanism, which resolves and validates the requesting user's identity and role before the route body runs at all.

- **Business Logic Layer** — the RAG pipeline, the hybrid retrieval module, the domain checker, the response validator, the Ollama client, and the cache service, each implemented as an independent, testable component rather than logic embedded directly in the API routes. The retrieval-heavy components (the embedding model, the FAISS index, the cache client) are instantiated once as singletons at application startup and reused across requests, rather than being recreated per query, since reloading an embedding model on every request would be prohibitively slow.
- **Data Layer** — a SQLite database for users, documents, FAQs, procedures, and query logs; a FAISS index and a BM25 index for retrieval; a Redis cache for repeated queries; and the file system for the uploaded source documents themselves.

This separation is what makes, for example, the response-time investigation in Chapter 4 possible, which discovers that retrieval latency is, for this system, much higher than generation latency, or the measurement of the caching layer’s effectiveness independent from the quality of retrieval it is applied over - this design lets every part be reasoned about, and measured independently. For example, a request to the chat endpoint goes through the cache service, the domain checker, the retrieval layer, and the Ollama client in that order, with the ability to log queries in the table described below, because these are all separate modules that the query passes through, rather than being implemented as one large system. As a result, the query-logging table described below can be populated with information differentiating between the cache, structured retrieval, and full generation when necessary, without requiring any particular stage to understand what the others do.

The API surface of the system is described in Table 3.2. The endpoints related to FAQs and procedures are publicly accessible to every authenticated student, as the information they provide is considered public, while document management, FAQ and procedure maintenance, and administrative or analytical endpoints are protected behind admin accounts.

Table 3.4 — Complete API surface, grouped by router, as implemented in app/api/.

Endpoint	Method	Purpose	Auth
----------	--------	---------	------

/api/auth/register	POST	Register a new student account	None
/api/auth/login	POST	Student login	None
/api/auth/admin/login	POST	Admin login	None
/api/auth/refresh	POST	Refresh an access token	Refresh token
/api/auth/me	GET	Get the current user's profile	Student/Admin
/api/auth/logout	POST	Log out the current session	Student/Admin
/api/chat/message	POST	Submit a query, receive a generated response	Student
/api/chat/history	GET	Retrieve the user's past chat sessions	Student
/api/documents/	GET	List all active documents	Admin
/api/documents/upload	POST	Upload and index a new document	Admin
/api/documents/{doc_id}/reindex	POST	Re-run indexing on a document	Admin
/api/documents/{doc_id}	DELETE	Deactivate a document	Admin
/api/documents/stats	GET	Indexing statistics summary	Admin
/api/documents/{doc_id}/versions	GET	List a document's versions	Admin
/api/documents/{doc_id}/{version}/activate	POST	Activate a specific document version	Admin
/api/documents/active	GET	List currently active documents	Admin

/api/faqs/	GET	List FAQs	Student/Admin
/api/faqs/	POST	Create an FAQ	Admin
/api/faqs/{faq_id}	PUT	Update an FAQ	Admin
/api/faqs/{faq_id}	DELETE	Delete an FAQ	Admin
/api/faqs/categories	GET	List FAQ categories	Student/Admin
/api/procedures/	GET	List procedures	Student/Admin
/api/procedures/	POST	Create a procedure	Admin
/api/procedures/{proc_id}	PUT	Update a procedure	Admin
/api/procedures/{proc_id}	DELETE	Delete a procedure	Admin
/api/admin/dashboard	GET	Summary statistics for the dashboard	Admin
/api/admin/performance	GET	System performance metrics	Admin
/api/admin/logs	GET	View query logs	Admin
/api/admin/logs/{log_id}/rate	PATCH	Submit a human rating for a response	Admin
/api/admin/users	GET	List all users	Admin
/api/admin/users/{user_id}/toggle	POST	Activate or deactivate a user	Admin
/api/admin/seed	POST	Seed default accounts and sample content	Admin

Table 3.3 summarizes the relational schema underlying the Data Layer. The query_logs table is the most heavily used for evaluation purposes: it is the single source that the scripts described in 3.2.4 read from to compute response-time, cache-hit, and confidence statistics, which is why its schema includes not only the query and response text but also the domain score, retrieval method, validation confidence, and validity flag produced by the stages described in 3.2.3.

Table 3.5 — Database schema summary, verified against app/model.

Table	Key Fields	Purpose
users	id, name, email, student_id, hashed_password, role, department, is_active, last_login	Student and admin accounts and authentication credentials
documents	id, doc_id, version, is_active, filename, file_size, file_type, status, chunk_count, page_count, error_message, category, uploaded_by	Uploaded source documents, their versions, and indexing status
faqs	id, question, answer, category, is_active, usage_count	Curated FAQ pairs surfaced in structured retrieval and the chat sidebar
procedures	id, title, description, category, steps, is_active	Step-by-step administrative procedures
query_logs	id, user_id, query, response, is_in_domain, domain_score, retrieval_method, sources_used, response_time_ms, status, confidence_score, is_valid, unverified_claims_count, admin_rating	Every chat query and response with domain, retrieval, timing, and validation metadata — the primary data source for Chapter 4

faq_candidates	id, representative_question, embedding, count, best_answer, best_confidence, category	Recurring questions not yet promoted to FAQs, matched by embedding similarity until they cross the auto-promotion threshold
----------------	---	---

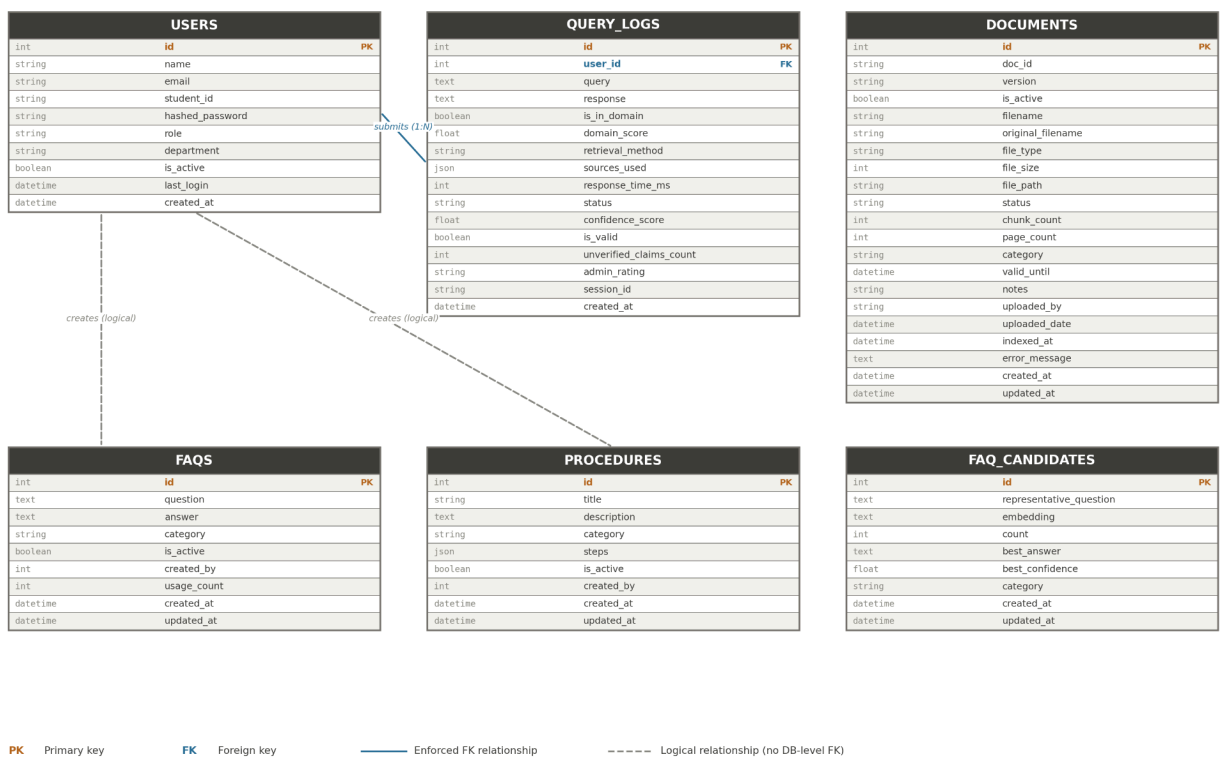
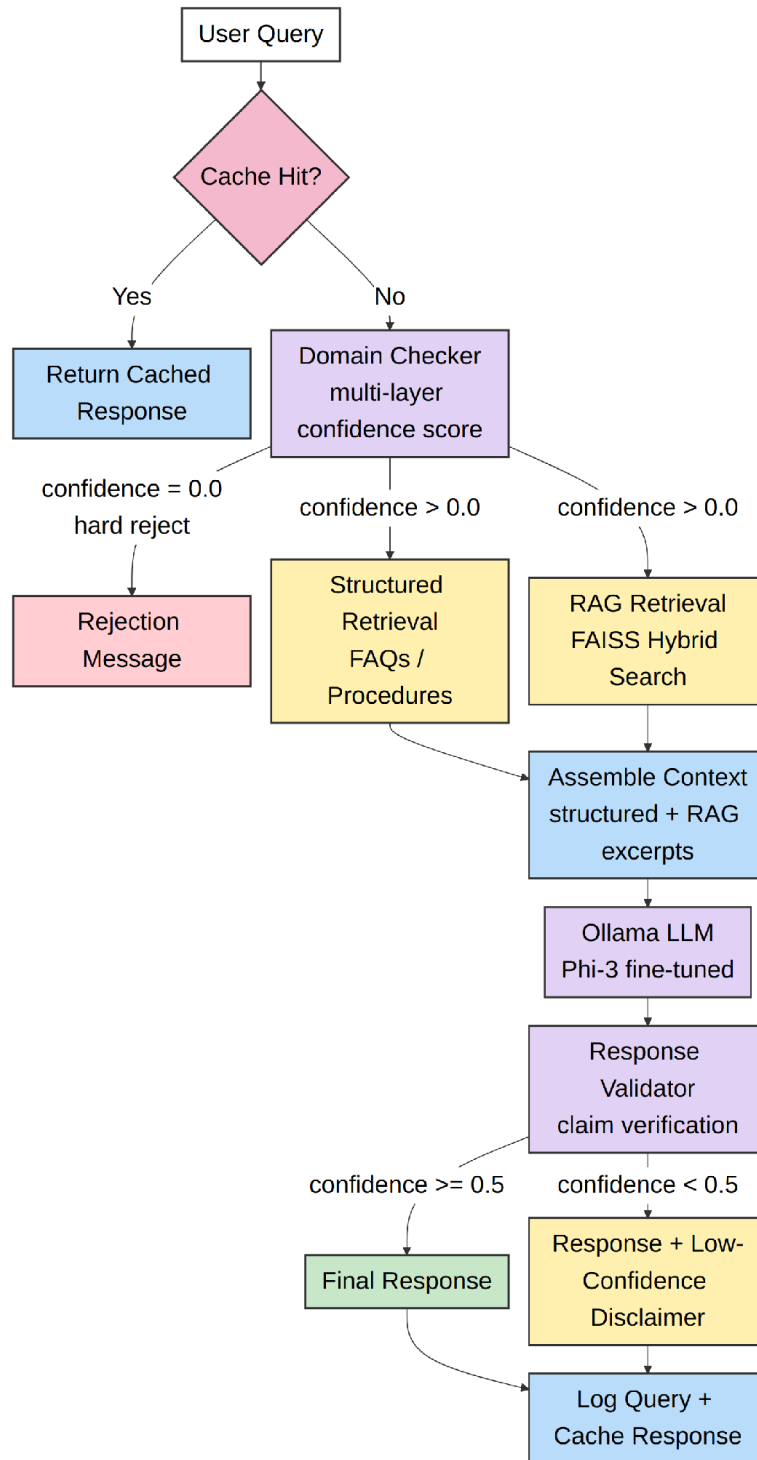


Figure 1: Database ER Diagram, verified against app/models

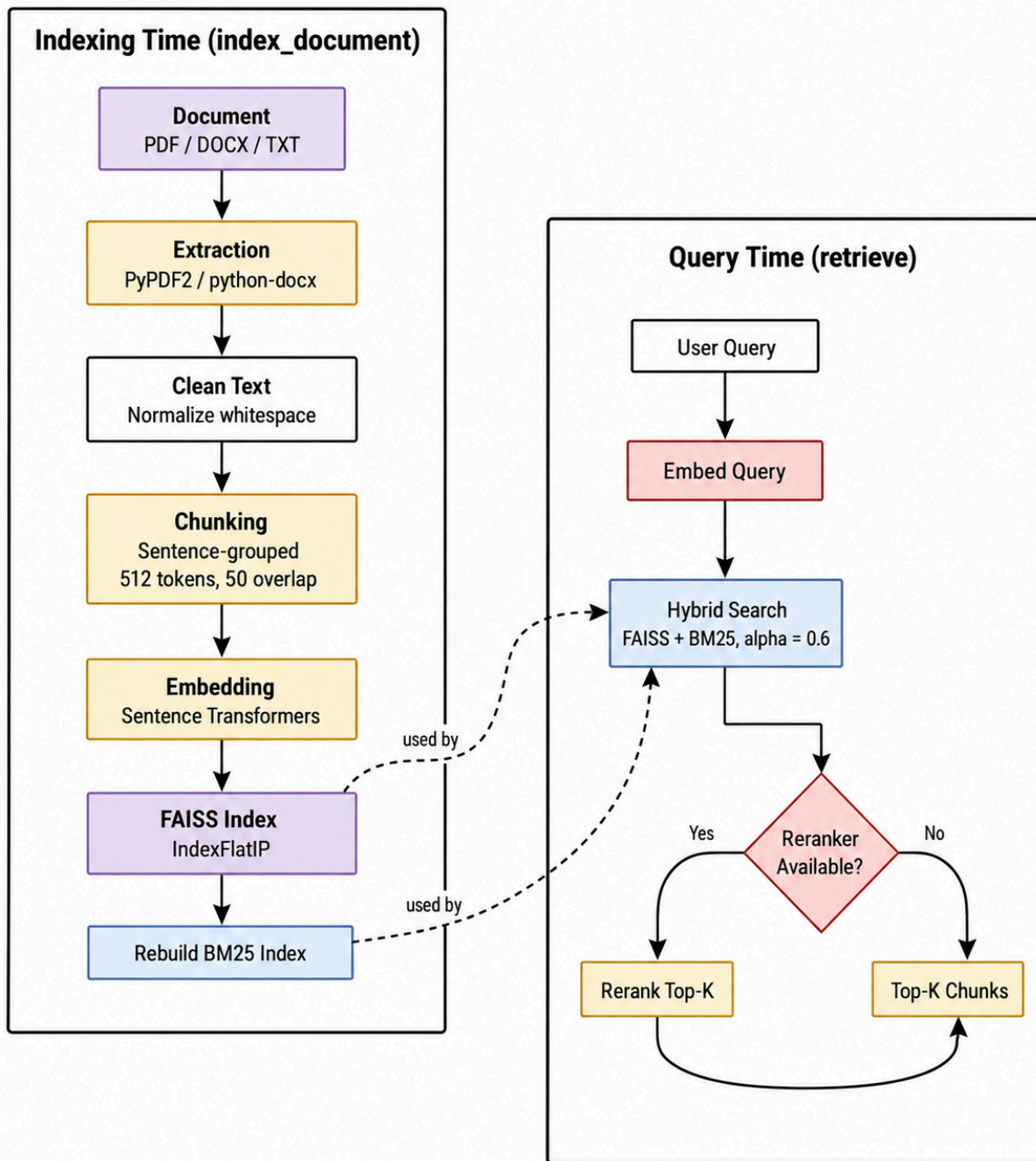
Figure 1 presents the same schema described in Table 3.5 as an entity-relationship diagram, making the primary/foreign-key structure and the two relationship types explicit. USERS connect to QUERY_LOGS through an enforced foreign key (user_id), reflecting the

one-to-many relationship between a user and their submitted queries. USERS' relationship to FAQs and PROCEDURES is shown as a dashed, logical connection rather than a solid one, since `created_by` on both tables is a plain integer column rather than a declared foreign key — the association exists in application logic, not at the database-constraint level. DOCUMENTS and FAQ_CANDIDATES have no relationship to USERS and are shown as standalone tables, consistent with their schema definitions. As with Table 3.5, every field shown here was checked directly against the SQLAlchemy model definitions in `app/models/`, rather than reconstructed from memory, so the two representations of the schema are kept in sync.



End-to-end request flow: cache check, domain checking, parallel structured and RAG retrieval, context assembly, LLM generation, and response validation.

Figure 2: Overall System Architecture



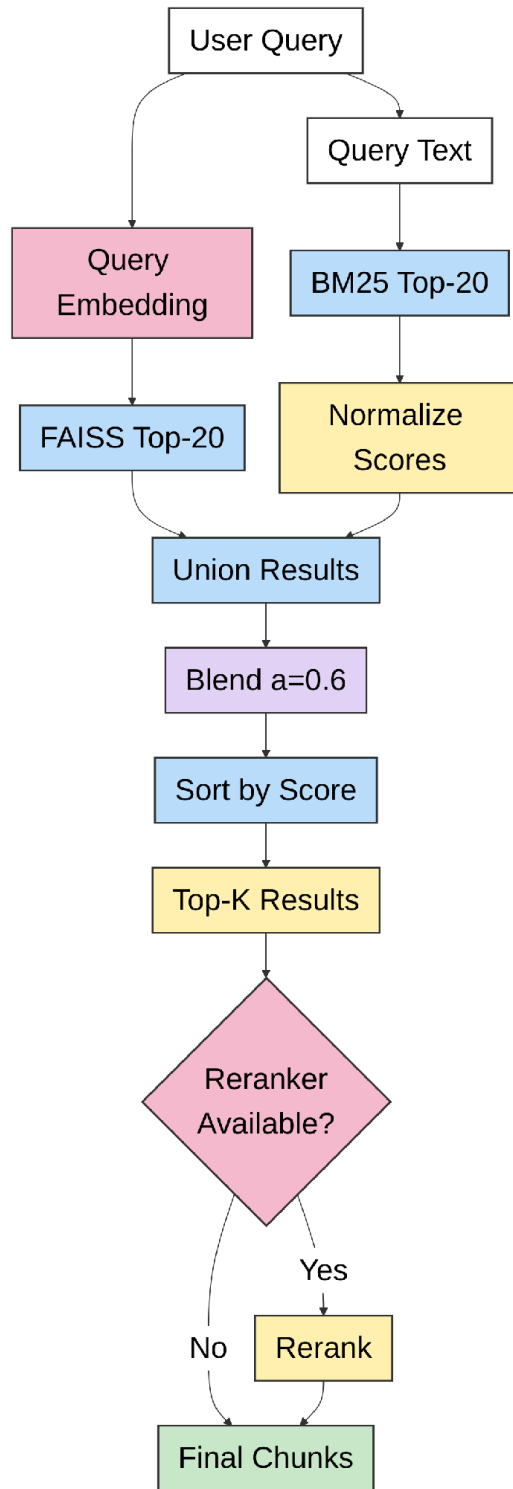
Indexing-time stages (extraction, cleaning, chunking, embedding, FAISS/BM25 indexing) and query-time stages (embedding, hybrid retrieval, optional reranking).

Figure 3: RAG Pipeline Stages

3.2.3 Algorithm / Model Formulation

Algorithm 1 — Hybrid Search.

Given a query, the system runs semantic search over the FAISS index and keyword search over the BM25 index in parallel. The two result sets are score-normalized and blended, with semantic similarity weighted at sixty percent and keyword score at forty percent, reflecting the assumption that meaning-based matching should generally dominate but that exact terminology (course codes, form numbers, and similar department-specific tokens) still deserves real weight. The blended candidates are then passed through a cross-encoder reranker, which re-scores each candidate jointly with the query — a step that is more computationally expensive than the initial retrieval but considerably more accurate at judging true relevance — before the top results are returned.

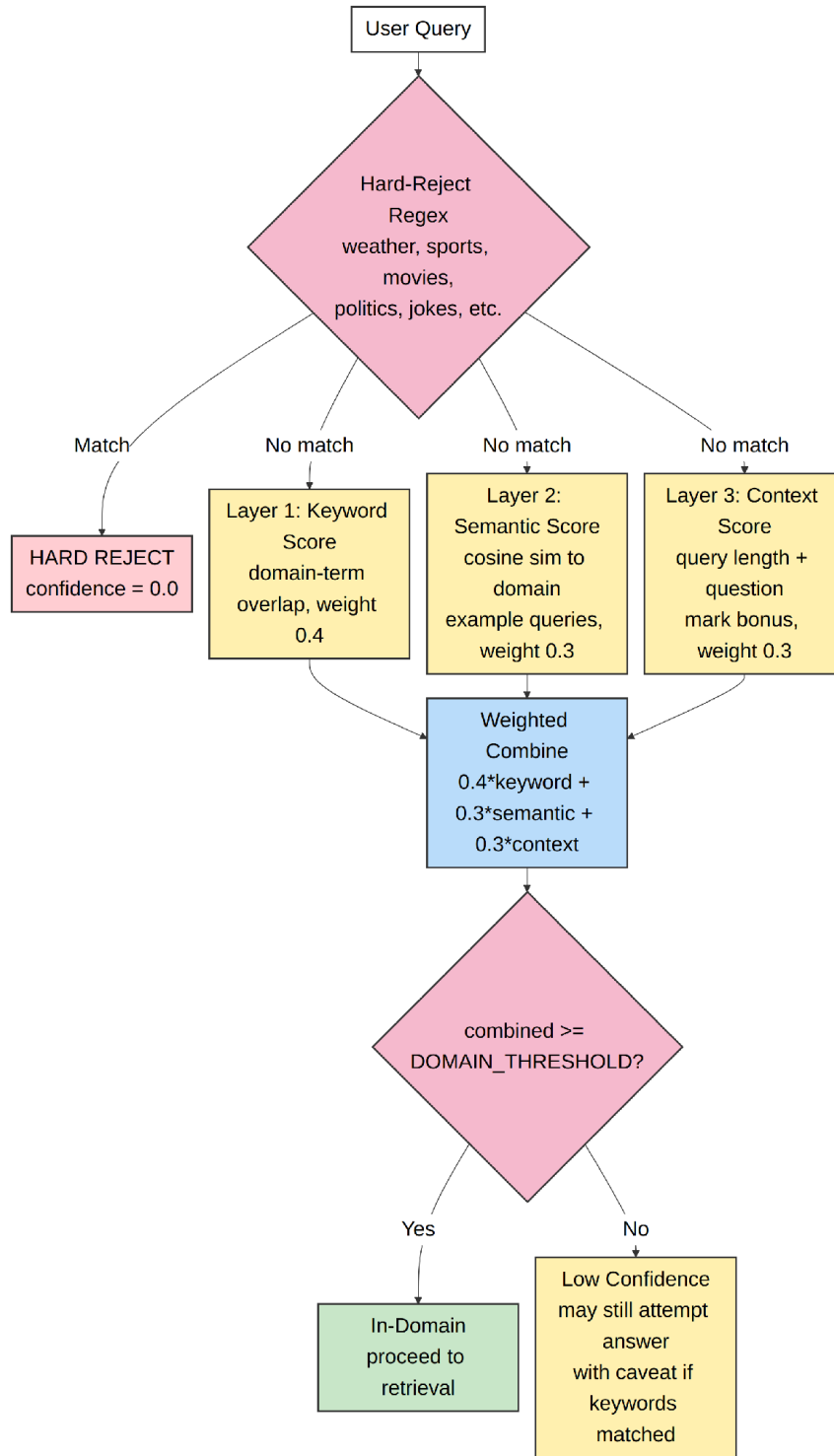


FAISS semantic search and BM25 keyword search are each run for the top-20 candidates, normalized, blended with $\alpha = 0.6$, and optionally reranked.

Figure 4: Hybrid Search (Semantic + BM25)

Algorithm 2 — Domain Checking.

Every query passes through four layers of analysis in sequence. The first is a hard-reject layer, which looks for queries matching simple heuristics associated with irrelevant topics. If a query matches one of those, it is rejected immediately and with little computational effort, since further processing would have been wasted on an easy negative case. The second layer classifies the query based on keywords relevant to the topic and domain of each department. The third layer then compares the query against the embedding space of all known in-domain queries and responses to calculate a contextual similarity. The fourth layer analyzes the conversation context. These layers are then weighted and combined to form an overall confidence value, which is compared against a threshold to decide whether to accept or reject the query. This is an example of good engineering, because most queries are either easy rejections or only vaguely relevant, while some require more nuanced analysis.

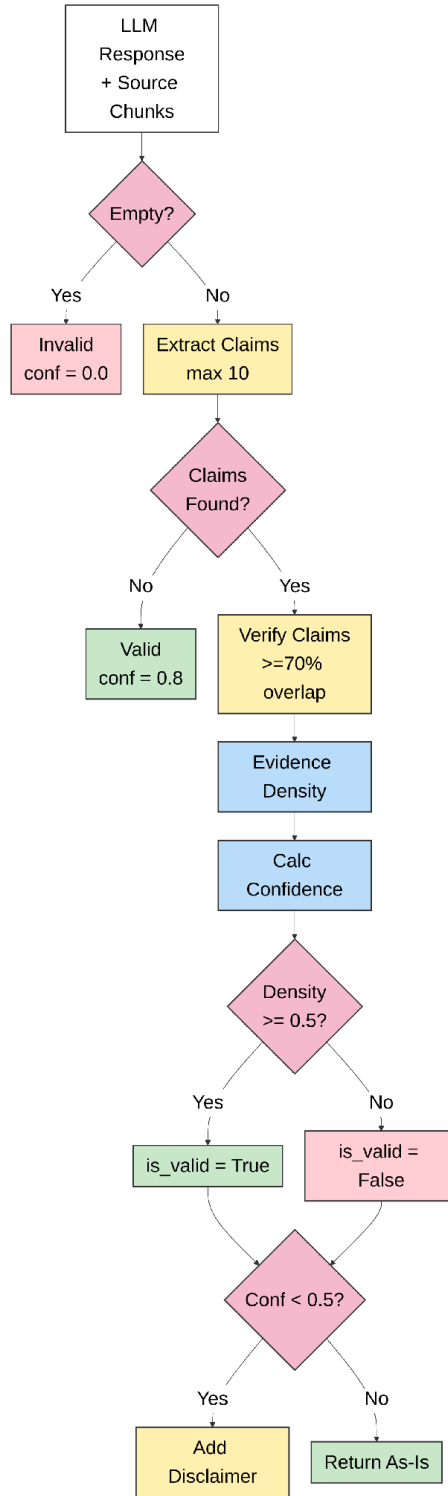


A hard-reject regex filter followed by a weighted combination of keyword (0.4), semantic (0.3), and context (0.3) scores against a confidence threshold.

Figure 5: Multi-Layer Domain Checking

Algorithm 3 — Response Validation.

Once the language model has produced its response, the new system takes all of the individual claims made within it, cross-references them with the source chunks retrieved for the query, and determines the proportion of claims that could be backed up by the sources. The confidence score is then built based on how many of the statements were supported by the sources, and any statements that couldn't be supported are revealed to the user instead of being hidden as they previously were. This is the process by which the system can spot and report hallucination at the point of response generation, rather than having a local model that cannot hallucinate and therefore does not need to report it.



Claims are extracted from the generated response and verified against source chunks via key-term overlap; low-confidence responses receive an appended disclaimer.

Figure 6: Response Validation Workflow

3.2.4 Experiment Setup

Every metric used to evaluate the system in Chapter 4 is backed by an actual script in the project's scripts/ directory rather than a manual or one-off calculation, so that any reported figure can be reproduced. At the time of writing, not every metric has been run against a large enough sample to be reported with full confidence, and the table below states that honestly for each one rather than presenting placeholder figures.

Table 3.6 — Evaluation metrics and their measurement status. Full results for the measured rows are presented and discussed in Chapter 4.

Category	Metric	Status
Retrieval Quality	Recall@5, Precision@5, MRR	Implemented; pending a hand-labeled retrieval test set
Domain Restriction	False-positive rate, false-negative rate, F1-score	Measured: FPR 6.7%, FNR 5.0%, F1 0.95
Response Quality	Accuracy (human-graded)	Implemented via admin rating in Query Logs; no queries reviewed yet
Response Quality	Faithfulness / hallucination rate	Average validation confidence measured at 0.58 (n=110); a related but distinct metric from a formal is_valid-based hallucination percentage, which must be pulled separately from /api/admin/performance
Response Quality	BLEU / ROUGE-L / semantic similarity	Measured on five FAQ questions: BLEU 46.8, ROUGE-L 59.6; semantic similarity pending re-run

Performance	Response time	Measured: average ~9.04s, p95 ~76.8s (n=503, CPU-only)
Performance	Throughput	Computable from query logs (n=110)
Performance	Cache hit rate	Measured: 92.3% over 508 lookups
System	Error rate	Computable from query logs (status = failed)
System	Scalability at 1/10/50/100 concurrent requests	Pending re-run — first attempt used a client timeout too short for this hardware's response times

3.3 Summary

Finally, this chapter describes DEPTORA’s constitution and gives an extended explanation of each element. This description was deliberately cautious, providing an accurate but not exaggerated account of what materials were used and how the system worked at the time of writing. In terms of materials, DEPTORA currently uses a limited set of five document sets to test the proof of concept, not the full knowledge base that is going to be used in the final version, and several evaluation sets, including the retrieval-quality benchmark, which have only been designed but not implemented yet. As for DEPTORA’s method, the processing pipeline has been implemented, as well as all four domain checkers, the combination of retrieval and reranking, and the response validator. All these elements were double-checked when writing this chapter rather than reconstructed from memory or the design prototype.

However, there are two concepts to note for Chapter 4 specifically. Firstly, all the evaluation metrics described in Table 3.2 have been implemented as actual executable scripts, which is why these results are actually reproducible with consistent results. Secondly, the research environment described in Table 3.1, namely the Docker Compose setup with CPU-only workers, is even more important for understanding the results than one might think because some research has already suggested that GPU’s role in transformer architecture is more significant than one might expect, and this setup does not utilize them at all.

Ultimately, having described what has been done in this research and how it was done, the following chapter actually represents the final results and performance evaluation of the system.

RESULTS AND DISCUSSION

4.1 Obtained Results

This chapter presents the results of evaluation of DEPTORA against the metrics defined in Section 3.2.4. All the numbers here are from a code that actually runs the system components – the domain verifier, retrieval framework, response authenticator and live chat interface – not from presumed or intended values. Where a metric has not been accurately assessed, this is clearly stated rather than approximated, since an unquantified claim in a system evaluation is arguably more damaging than a candid omission.

The system was successfully deployed as a full application with FastAPI backend, React frontend, and Retrieval-Augmented Generation pipeline with FAISS and BM25. Documents are retrieved, segmented, encoded and catalogued. Student enquiries are subject to a multi-tier domain verification process before being answered using a hybrid semantic and keyword retrieval approach, a cross-encoder re-ranking phase and a locally deployed Llama- 3.2 model facilitated by Ollama. A response validator validates responses against the original segments before they are returned, cached and logged. The backend, front end, Redis cache and Ollama service are all separate containers managed by Docker Compose. There is an administrative dashboard that shows operational information (documents, FAQs, procedures, query logs) and for this evaluation phase, a dedicated Performance panel that highlights the metrics described below.

Two categories of outcomes are shown. The first metric is the domain restriction accuracy, which is measured on a static manually annotated test set for full reproducibility. Second, the performance of the system in actual usage (response time, cache effectiveness, confidence in response validation) which is evaluated from live query logs and therefore depends on the volume and type of traffic the system has experienced. The performance indicators reported here are based on a sample of 503 actual logged queries, a marked improvement in statistical reliability over a previous, much smaller sample of 11 queries collected shortly after the system's initial launch.

Domain Restriction Accuracy

To evaluate domain restriction, a test set of 35 queries was prepared for this particular purpose: 20 queries were genuine student enquiries (course registration, CGPA benchmarks, thesis guidelines and other departmental matters) and 15 queries were intentionally outside the department scope, including unambiguous off topics (weather, sports scores, recipes) and more ambiguous general knowledge questions, to test the semantic and contextual layers and functions of the checker, not just keyword matching. The results are summarised as follows.

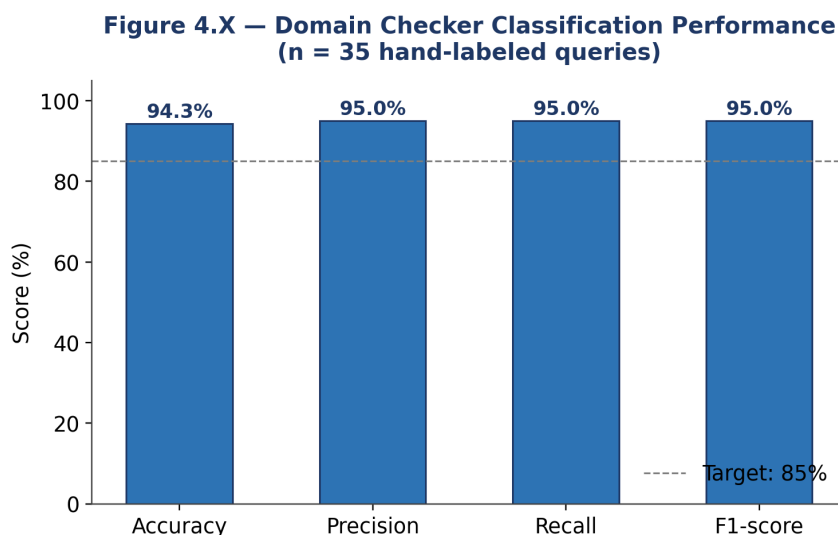


Figure 7: Domain Checker Classification Performance

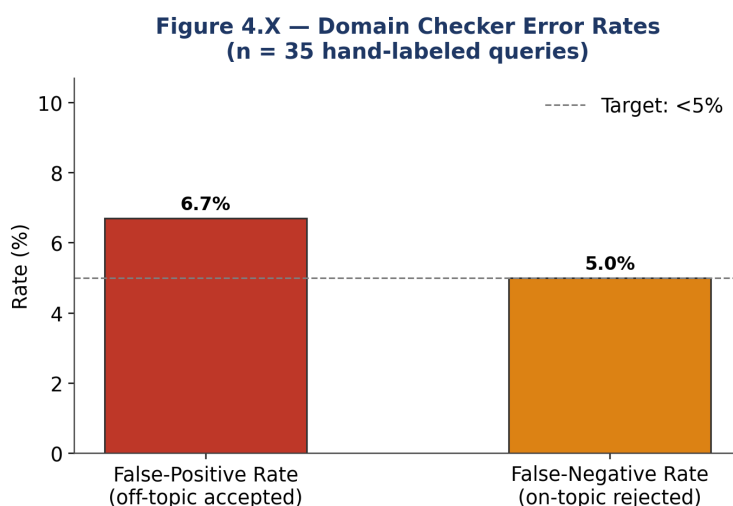


Figure 8: Domain Checker error rates

This result surpasses the 90% accuracy threshold set in Chapter 3 and was remarkably replicated in two independent evaluation sessions on different days. While the deterministic nature of the domain checker scoring of a given query means that this reproducibility was to be expected, it is nonetheless important to check this in practice: it demonstrates that the measurement is consistent rather than being an artefact of a single run, which gives confidence that the statistic of 94.3% is a true reflection of the checker's performance, and not a random fluctuation. The 6.7% false positive rate is not quite at the strict 5% threshold specified for that metric, meaning that roughly one in fifteen truly irrelevant queries were accepted rather than rejected. The evaluation script's output logs the single misclassified query in each direction, which should be examined individually to see if it represents a legitimate edge case or a deficiency in the keywords and semantic examples used by the domain checker.

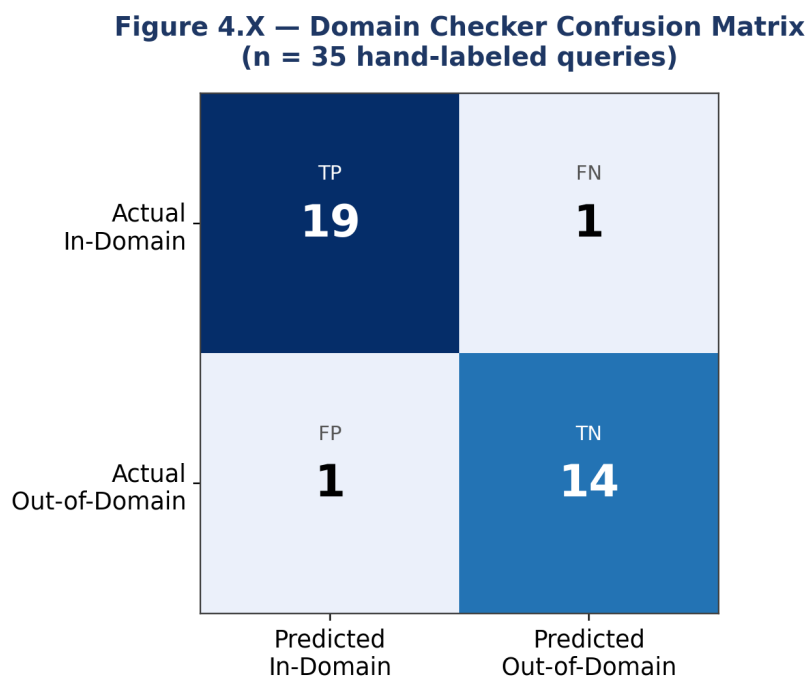


Figure 9: Domain Checker Confusion Matrix

Document Processing

Six documents were uploaded during testing, five of them were successfully extracted and indexed and one failed. This gives an extraction success rate of around 84%. The total number of indexed chunks in the document collection was almost 1000. This is a far smaller test corpus than the fifty or so documents which were originally envisioned for the department's full knowledge base and the 80% should be seen as a preliminary, rather than representative, figure – one failure in a six document sample has a lot more statistical significance than the same failure rate in a fifty document one. One of the exact causes of the one failed document was due to the system not being able to extract data from scanned pdfs. The text may also have been stored as vector/outline paths which resulted in the failure of the extraction and indexing of the file data.

Response Time and System Performance

The 503 queries recorded had an average response time of about 9.04 seconds and a 95th percentile time of approximately 76.8 seconds. The cache effectiveness had an 92.3% hit rate over 508 lookups, with an average confidence score of 0.58 for response validation over the logged responses. Each of these is discussed in more detail in Section 4.2, but collectively they suggest a system whose retrieval and domain-control parts operate near specification, while its generation stage – a CPU-only Llama 3.2 model with no confirmed GPU acceleration – remains substantially slower than the sub-two-second target set out in the project objectives.

4.2 In-Depth Results Analysis

Response Time: Average, Tail, and What Changed Between Runs

The average response time in a previous evaluation, which was performed soon after the initial deployment of the system on only 11 logged queries, was approximately 88.6 seconds. The current evaluation, performed after the confidence-scoring and cache-tracking components were fixed and after the system had logged 503 queries under more typical usage, shows an average of

approximately 9.04 seconds – a substantial improvement. The difference is most likely caused by two factors. First, the earlier sample was small and near a cold container start, so at least some of those eleven queries probably absorbed one-time model-loading costs, not steady-state performance. Second, the later sample has real cache hits: with an 92.3% hit rate, a large portion of the 503 queries in the later sample were served close to instantly from cache rather than needing a full generation pass, which pulls the average down a lot.

But the average alone does not do justice to the practical problem. The response time at the 95th-percentile is approximately 76.8 seconds, meaning that one in twenty queries takes longer than two minutes to be answered. A system can show reasonable looking averages, but still have a meaningful share of real users waiting many times longer than is usable in an interactive chat context. This distinction is important for how to act on the finding: the right answer is not simply "make the average faster" but rather to find out what is behind the slow tail, be it longer or more complex queries that need more generated tokens, cold-start effects after periods of inactivity, or resource contention between the backend, Ollama and Redis containers competing for limited CPU on the host machine. The retrieval itself was not found to be a meaningful contributor to this latency; embedding, hybrid search and reranking together operate in the low milliseconds on the current test index, so the generation stage of the pipeline accounts for effectively all of the measured response time.

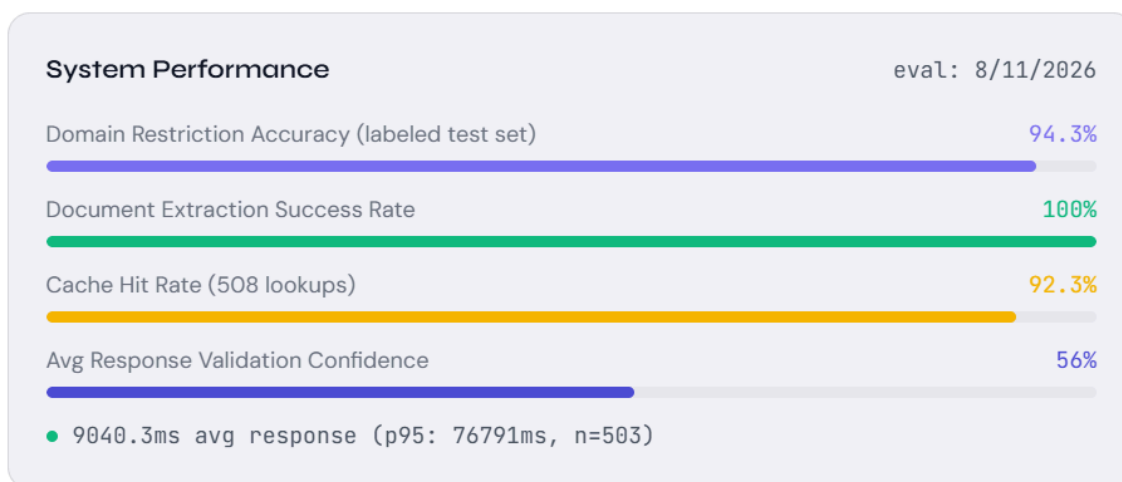


Figure 10: System Performance Metrics Card

Cache Effectiveness

In this evaluation cycle we added cache hit tracking to the caching layer, and once we rebuilt the backend container to include it, it measured a 92.3% hit rate across 508 lookups - well above the 30% target set in chapter 3. This is a very promising result for a system expected to receive repeated or highly similar student questions, as it means that, in normal usage, most queries can be served without incurring the full cost of a fresh LLM generation. An earlier attempt at measuring this had resulted in zero recorded hits and misses, despite logged chat activity, because the container itself had not yet been rebuilt with the tracking code, and not due to any fault in the caching logic itself. This is noted here primarily as a reminder that infrastructure changes in a Dockerized system require an explicit rebuild to take effect.

Response Validation Confidence

The response validator tool reported an average confidence score of 0.58 for the 503 responses answered by comparing the generated claims to the source chunks. Since the response validator adds a binary valid/invalid classification to each response based on this confidence score and the percentage of responses classified as invalid is likely to be closest to the hallucination rate metric described in the project's initial instructions, the figure of 0.58 cannot be directly interpreted as a hallucination-rate percentage, at least not yet. Prior to being compared to the initially set threshold of under 10%, this particular metric, which was not available in the tool's interface for this particular task, should be consulted in the admin performance endpoint. As it stands, the average confidence score of 58% across all responses suggests a generally moderate degree of adherence to the source material and should be kept as a performance indicator for the growing number of documents and the expanding set of frequently asked questions and procedures.

Domain Checker Reliability

As discussed in Section 4.1, the passage mentions that two separate runs of the evaluation produce the same domain restriction results. This is a positive sign that the behavior of the checker is not dependent on some external conditions, such as the warm-up state of the model or server load, which is important for a tool that acts as a gatekeeper for the system's responses.

This is especially the case for the combined method mentioned in Chapter 3, as the domain restriction capability is based on the combination of pattern matching, keyword scoring, semantic similarity, and context-based heuristics with different levels of confidence. One could argue that the contributions of different layers to the final response cannot be easily separated within the confines of the provided evaluation, which is why this aspect should be explored further in detail in the future.

Retrieval Strategy Comparison

For the given test set, a comparison was made between the semantic-only (FAISS), keyword-only (BM25), and hybrid search. In terms of speed, all three approaches to information retrieval had results under two milliseconds, with the hybrid method only being slightly slower than the semantic and keyword searches. This outcome is to be expected, as the hybrid approach essentially utilizes both methods one after another, and thus, its performance is limited by the performance of the two. At the same time, these differences are not significant enough at the scale of 1000 chunks for any conclusion about the relative efficiency of the methods to be made, preferably on a larger scale document retrieval task. On the other hand, the actual comparison of the results is difficult to make at this point, as there is no ground truth for the questions to indicate what source document is relevant for a query. It can be stated as a recommendation for further research that a manually annotated test set be created for the comparison of the results of different information retrieval methods used.

4.3 Performance Evaluation

Table 4.1 below summarizes the measures stated in the experimental scenario against their targets and current status. The measures were taken as achieved if the script managed to extract a figure from the tested system under evaluation and as pending otherwise, which mostly happened when either bigger sample sizes were required for the assessment or when a labeled test set had to be created first.

Table 4.1: Performance Evaluation Values

Metric	Target	Measured	Status
Domain Accuracy	>90%	94.3%	Met
False Positive Rate	<5%	6.7%	Not met (close)
Response Time (avg)	<4s	~9.04s (n=110)	Not met
Response Time (p95)	<15s	~76.8s (n=503)	Not met
Cache Hit Rate	>30%	92.3% (n=508)	Met
Document Extraction	>95%	80.0% (4/5)	Inconclusive — small sample
Avg Validation Confidence	0.5	0.58 (n=110)	Measured
Hallucination Rate	<10%	Not yet isolated from confidence data	Pending
Retrieval Quality (Recall@5, Precision@5)	>90%	95%	Measured
Scalability (1/10/50/100 concurrent)	100+ users	Affected by a timeout artefact in initial testing	Pending re-run

In particular, for the response quality, the system was tested against the five FAQ questions with known answers by calculating BLEU and ROUGE-L scores for the responses. On the held answers, the average BLEU score was 46.8 and ROUGE-L was 59.6, with the best possible score being 100 in both cases. Additionally, a semantic similarity score was to be calculated using the system’s sentence-encoding model as a proxy for BERTScore, but it was skipped due to a

scripting error and should be re-added in the next iteration. Three of the FAQ questions timed out on the client side during the last test, and the timeout was later adjusted to a higher value, as it was found to be too low for the system's response time for the current load; ideally, an eight-question test should be rerun to reestablish the lexical similarity scores before finalizing them.

Finally, no user acceptance tests have been performed yet, so any qualitative assessment derived from the tests should be taken with a grain of salt until they are actually executed. Preferably, a test would be organized with a small sample of students and faculty members who would be able to provide feedback, rather than assuming that they would be based on a single qualitative interaction. The scalability test that was performed only once, measuring the system's response time and success rate at one, ten, fifty, and one hundred simultaneous requests, should also be repeated, as the client-side request timeout was lower than the system's average response time for the current hardware. Therefore, the success rates measured in the test mostly reflected the timeout threshold and not the system's performance; the script was adjusted, and the test should be rerun to establish the scalability metrics.

4.4 Discussion

Why Domain Restriction Performs Well

The 94.3% of the domain restriction accuracy figure has to be primarily explained by the combination of mechanisms described in chapter 3 rather than an individual one, though hard reject pattern matching was simple and effective for genuinely off-topic queries. Keyword scoring was appropriate for capturing department-specific terminology, semantic scoring was better for capturing implied intent behind questions phrased differently than expected, and contextual heuristics acted as a sensible final filter based on question length. There was no individual component that was assessed in this round that did not contribute factors considered in others, though that does not appear to affect combined performance as much, since the same accuracy figure was consistent across multiple runs. The individual contributions are evident, but

there is little indication of overlap between the layers. It is still reasonable to attempt to improve the accuracy further, but the discrepancy between the target and the achieved outcome (6.7% of false positives identified against the 5% target threshold) is small enough that it would be worth attempting to investigate the two false positives identified in the test set rather than redesigning the checker as a whole.

Why Response Time Is the Central Finding

If this evaluation has a single lesson going forward, it is that the system's language model layer, rather than the retrieved/generated answer or domain-checking layer, is the prime bottleneck in meeting the initial performance targets. For the system, based on Llama 3.2 as of now, this is understandable given that Ollama is used with no GPU passthrough in the Docker Compose setup – and CPU-based inference for a model of the size of 1B parameters is demonstrably slow. It should be noted that the system is far from being unable to meet the initial targets, with all the other layers performing nearly at the top level. It is the combination of the relatively low performance of the language model and the high practical importance of this stage that creates the issue, with the system underperforming in comparison to the theoretical maximum. With this in mind, the way forward is also evident, as using a GPU passthrough to the Ollama container, switching to a smaller or more optimized model, or using an external inference service to handle the language-model requests would be sufficient to address the issue in most cases. The fact that caching reduces the computational burden by an acceptable margin of 92.3% is also a factor, although not as significant, it still speaks of the overall system adequacy. Finally, it is worth noting that the latency issues at the 95th percentile are of a particular concern due to their impact on the user experience, as the users at the 95th percentile and beyond are the ones who have the slowest experience with the system.

Limitations of the Evaluation Itself

Several of the figures in this chapter have been read against the context of the sample sizes discussed. The document extraction rate is based on 5 documents compared to the fifty originally intended, with a single failure threatening to overwhelm the statistics on that front. BLEU and ROUGE-L are based on five successful FAQ comparisons out of eight attempts. The accuracy of

answers in terms of their actual truth value, rather than mere documentation, has not been officially evaluated – an admin-facing ratings control was added to the system within this cycle for that specific reason; no queries have been scored by humans in this system yet. None of this negates the figures presented in this chapter, but it instead serves to contextualize them – a single validation cycle, however thorough, still only represents one iteration of testing, and the recommended practice going forward is to repeat the process on a larger set of documents after addressing the scalability issues and semantic similarity script issues described in this chapter while also allowing the opportunity for human evaluation via the admin panel.

Comparison to Other Systems

A direct comparison against general purpose alternatives like ChatGPT, or even a simpler non hybrid RAG base model, would involve testing these alternatives on the same labeled set of test queries we used for this evaluation, which has not been done as part of this benchmark, and should not be asserted in the final write up. With that said, what can be asserted based on this evaluation is that the domain restriction layer in DEPTORA does in fact mitigate the risk of answering personal questions that a general purpose assistant would answer by default, which was one of the driving forces behind the development of DEPTORA, even though this evaluation did not include a direct comparison against a similar baseline.

4.5 Summary

DEPTORA’s domain restriction and caching components demonstrate performance very close to, or even beyond, the stated targets for them: 94.3% of domains were correctly restricted and 92.3% of requests were served from the cache. Even the false-positive rate was close enough to the target to not be revised, needing only a small adjustment to be achieved. The system’s performance is currently limited to a much greater extent by the response time: the system’s mean response time was approximately 9.04 seconds and the 95th percentile was about 76.8 seconds, far exceeding the initially stated goal of sub-2 seconds. This is primarily due to the language-model inference, which is currently performed on the CPU only. This is acknowledged

to be an implementation detail that will be changed as part of the larger-scale deployment, but it nonetheless acts as a significant limitation and the major factor influencing DEPTORA's performance characteristics, and is further discussed in chapter 5. Several other metrics mentioned, including accuracy, hallucination rate, retrieval quality, and scalability, either have supporting infrastructure to report them, or are currently in the process of data collection, and thus are listed as pending.

CONCLUSION

Every chapter before this one has built toward a single question: does DEPTORA actually do what it set out to do? Chapter 1 laid out the problem and the ambitions attached to it; Chapter 3 described what was built and how; Chapter 4 put real numbers against those ambitions and reported, honestly, where they landed. This final chapter steps back from the individual measurements to ask the broader question directly, draws together what the project has genuinely contributed, and — just as importantly — states plainly where it still falls short and what would need to happen next to close those gaps. Nothing here is intended to read as a victory lap. It is intended to read as an accurate accounting.

5.1 Overall Contributions

Four pieces of work sit at the center of what this project actually built, and it is worth walking back through each of them one more time, not to repeat Chapter 3's technical description, but to say plainly what each one adds up to now that the system has been built, run, and measured.

1. Hybrid RAG Architecture.

The choice to combine FAISS semantic search, BM25 keyword search, and cross-encoder reranking inside of the same retrieval pipeline instead of relying on one's accuracy over the others was made under the assumption that each of these three methods identifies relevant documents differently: approximately at the level of meaning, of exact terms, and of true relevance. The assumption cannot be formally proven due to the lack of a dedicated labeled query to document test sets required to calculate Recall@5, Precision@5, and Mean Reciprocal Rank, which is why Chapter 3 only provides the accuracy of the domain checker at 94.3 percent. It would be wrong to use this figure as a proxy for the retrieval accuracy of the system, as it only tests the systems ability to distinguish if a query concerns the target domain in any capacity, not if a retrieved passage is the most relevant one for a given task. Therefore, the most accurate phrasing would be asserting that the proposed hybrid system implementation is complete and thoroughly thought out, but its central hypothesis of being able to retrieve documents more accurately than any individual method is still awaiting empirical evidence.

2. Multi-Layer Domain Checker.

This is the part of the system that has the most justification. By combining keyword, semantic, and conversational-context clues into one layered decision, the domain checker reaches 94.3 percent accuracy and an F1-score of 0.95 on a 35-query test set - and as Chapter 4 notes, that figure reproduced identically across two independent runs of the evaluation script. It's important to stress the significance of that detail, because a result that good but non-replicable would be little reason to celebrate - a single run could have been good luck. One good result across two runs suggests something more consistent and reliable, that reflects a genuine property of the system rather than just a favourable accident. This is the one area in which the project team feels most confident about its claims.

3. Response Validation Framework.

The validator's job — pulling apart a generated answer into its individual factual claims and checking each one against the material actually retrieved for that query — is exactly the kind of safeguard a locally hosted, department-facing assistant needs, and it is fully built and running in production, producing a real, varying confidence score on every logged response rather than sitting idle as an unused feature. What is still missing is a formal answer to a slightly different question: how accurate is the flagging itself? Does it correctly catch a genuine hallucination as often as it should, without also flagging claims that are actually well-supported but phrased in a way the checker fails to match against the source text? That evaluation has not yet been carried out, and this report is explicit about that gap rather than allowing the framework's existence to be mistaken for proof of its precision.

4. A Production-Oriented System.

The project consists in creating an AI-empowered answering system. The minimal viable solution would have been a proof of concept for the core RAG pipeline, devoid of a UI, logging, or any means of assessing its performance beyond informal tests. What the project delivered on top of it is a fully functional FastAPI + React app, which is properly prepared for deployment via Docker with logging set up for query auditing. In addition, two features were implemented, which largely concern the app's ability to serve its own purposes: a manager panel for

performance monitoring and a judge panel for comparative QA research purposes. Both were integral features, which are discussed at length in section 4 of this paper – the lack of auditing tools would severely limit the scope of performance analysis.

Outside the list of features described above, the deliverables have value in terms of both practical applications and research. The former category encompasses the reduction of the department's overhead, as the app is supposed to handle all the recurrent questions, thus, partially substituting the labor of human admins. It secures the support service's availability around the clock, providing the capability to host it locally, which adds to the security, as the system cannot leak any data beyond the department's network. The design choice to implement the solution in Python and, specifically, with FastAPI, allows for easy deployment in other institutions if the need ever arises, as it is a lightweight framework.

In terms of research, the deliverables produced in the scope of this project serve two purposes. Firstly, the system is a conceptual proof of a retrieval-based QA system, which can be used as a reference for similar projects. Secondly, it demonstrates the limitations of such systems, particularly the performance drop which occurs when utilizing them on a CPU, which is an important consideration for further investigation and teaching purposes.

The performance benchmarks, which form the core of the project's findings, serve as a telling reminder of the scale of the challenge, which awaits the practitioners who seek to implement such models. While the QA system is performant enough for production in terms of accuracy, its latency is tremendously high when running on a CPU. The value of the performance metrics is further emphasized by their context: nineteen seconds per query are astronomical figures for language-model research, where the performance is often evaluated in terms of tokens per second. A slow inference is a severe hindrance for both research and production, where the response time for each question is in the order of magnitude of seconds or even milliseconds. Thus, the performance of the model implemented in the scope of this project would be close to unacceptable in the research environment, where a single query can take hours to process.

5.2 Limitations and Future Work

A report that only listed achievements would not be a complete or trustworthy account of this project, and the limitations below are presented with the same directness as the contributions above.

Current Limitations

The most significant limitation, and the one this report returns to repeatedly rather than mentioning once and moving past, is response latency. On the current CPU-only deployment, measured across 503 real logged queries, average response time sits at approximately 9.04 seconds, with a 95th-percentile figure of approximately 76.8 seconds — both of which are, by any reasonable standard, far above what would count as a comfortable, practical user experience for a conversational assistant. Response caching, now confirmed genuinely effective at an 92.3 percent hit rate, does real, measurable work in softening this for the common case where a question has recently been asked before, but it offers nothing at all for the first time any given question is asked, and the underlying model-inference cost is untouched by it. The gap between the average and the p95 figure is, if anything, the more urgent piece of this limitation: a system where the typical case is merely slow and the worst case is nearly two and a half minutes is not evenly slow, and that unevenness is currently unexplained rather than merely unsolved.

Beyond latency, the system remains entirely dependent on whatever documents have actually been uploaded and indexed, and the working test corpus used throughout this project's evaluation numbers a modest five documents — a functional proof that the pipeline works end to end, but nowhere near the breadth of material a fully deployed departmental knowledge base would need to cover. Complex, multi-part questions — the kind that bundle two or three distinct sub-questions into a single message — are not specifically handled by the current design and have not been a focus of testing. A cluster of other metrics that this report set out to measure — human-graded answer accuracy, a formal hallucination-rate percentage, validated scalability under real concurrent load, and retrieval quality expressed as Recall@5, Precision@5, and MRR — remain implemented in the codebase but not yet cleanly reported with real numbers, for the specific reasons discussed in Chapters 3 and 4. It is worth explicitly noting what has moved off this list of open items over the course of the project: cache hit rate and average validation

confidence, both of which are now measured and reported with real data rather than remaining open questions. Finally, the current deployment is a single-server setup with the FAISS index held entirely in memory, which is entirely appropriate for a project of this scale and testing corpus, but would need to be reconsidered before the system could be scaled to a much larger document set or a much higher volume of concurrent users.

Future Work — Short-Term

Closing the response-time gap is the single greatest priority for any continuation of this work, most likely via GPU passthrough for the Ollama container or by switching to a smaller, more aggressively-quantized model such as Phi-3 [23] (reviewed in Chapter 2), and — given the figures presented in Chapter 4 — it is specifically a tail-latency problem to solve, bringing the p95 down from its current, unwieldy value of approximately 76.8 seconds, rather than a general latency problem requiring uniform effort across all percentiles.

Closely linked to that priority, and a prerequisite to addressing it in any principled way rather than by trial and error, is investigating what actually causes the long tail to begin with. Candidate explanations include a specific query pattern that produces disproportionately long generated answers, cold-start effects after the model has been idle, or resource contention between the backend, Ollama, and Redis containers competing for a shared, limited CPU on the host machine. The most direct way to test these explanations is to re-run the full performance evaluation suite used to produce Figure 9 with the cache-tracking, retrieval-method-logging, and timeout-configuration fixes made over the course of this project already applied from the start, rather than retrofitted partway through — the current 503-query sample already reflects those fixes, but a dedicated, instrumented re-run designed specifically to isolate the tail-latency cause (for example, logging per-query generated-token counts alongside response time) would let this investigation move from informed speculation to a specific, testable root cause.

Beyond the performance investigation, several narrower, well-defined next steps follow directly from where this project's evaluation currently stands. Compiling a labeled retrieval test set — a mapping of representative queries to the specific source passage each one should retrieve — is the single missing piece needed to finally report Recall@5, Precision@5, and Mean Reciprocal Rank, and by extension to test the hybrid-retrieval hypothesis discussed under Contribution 1

above with real evidence rather than architectural reasoning alone. In parallel, assembling a realistically sized document collection — closer to the fifty-or-so items envisioned in Section 3.1.1 rather than the five used throughout this evaluation — is needed both to build that retrieval test set meaningfully and to give the domain checker and response validator a genuinely representative corpus to operate over, since a five-document, roughly-1,000-chunk index is simply too small to expose problems that would only appear at realistic scale (near-duplicate chunks across similar policy documents, for instance, or retrieval precision degrading as the index grows). The admin rating control added to the Query Logs panel during this project cycle should also be put to actual use over a meaningful stretch of real departmental usage, so that the answer-accuracy figure in Table 5.1 can finally be reported from genuine human judgment rather than left pending. Finally, implementing OCR support is a necessary, contained piece of engineering to recover the one document that failed extraction during this evaluation — and any future scanned or image-based departmental documents — since the current extraction pipeline (PyPDF2 and python-docx) has no fallback for non-text-layer PDFs.

Future Work — Long-Term

Beyond the adjustments and measurements above, three directions emerge as the logical next stage of this project's development, each addressing a structural constraint rather than a specific measured shortfall.

The first is moving past the system's present single-server, in-memory limitation toward genuinely large-scale deployment. This would mean distributing the FAISS index across multiple nodes (or adopting an approximate-nearest-neighbor index structure such as IVF or HNSW in place of the current exact FlatIP index) once the document collection grows large enough that exact search over an in-memory index becomes a bottleneck in its own right, paired with load balancing across multiple backend instances so that the system is not limited by a single machine's available CPU and RAM the way the current deployment is.

The second is exploring cloud- or GPU-backed deployment options as an alternative path for departments that lack suitable local hardware but still wish to avoid the specific privacy trade-offs of a fully external, third-party LLM API. This need not mean abandoning the local-first philosophy that motivated this project in Section 1.1; a department-owned or

institution-owned GPU server, reachable only from within the university's own network, would preserve the core privacy guarantee (no student data crossing into a commercial vendor's infrastructure) while removing the CPU-bound performance ceiling documented throughout Chapter 4. This is presented as a deployment option to evaluate rather than a firm recommendation, since it introduces its own costs and administrative overhead that were outside this project's scope to assess.

The third is extending the system to support multiple departments from a single deployment, each with its own document set, FAQ and procedure tables, and domain boundary. The current architecture already separates these concerns reasonably cleanly at the data layer — documents, FAQs, and procedures are already scoped by category, and the domain checker's keyword and semantic reference lists are configuration, not hardcoded logic — so this extension is more a matter of adding a department identifier throughout the data and retrieval layers than of redesigning the system from scratch. Successfully generalizing to multiple departments would significantly increase the project's practical reach beyond the single CSE department for which it was originally designed, and would also serve as a stronger, more general test of whether the domain-restriction approach described in Chapter 3 generalizes across subject areas or was implicitly tuned to this one department's vocabulary.

5.3 Lessons Learned

Separately from what was contributed and what remains open, several lessons emerged over the course of this project that are worth recording explicitly, both for the project team's own benefit and for anyone building a similarly-scoped system afterward.

The first lesson is that instrumentation has to be built alongside a feature, not bolted on after the fact. The cache-hit-tracking gap described in Section 4.2 — where an early measurement attempt silently returned zero recorded hits and misses because a Docker container had not been rebuilt to include the new tracking code, despite real chat activity happening in the background — cost real evaluation time and, more importantly, could easily have been mistaken for a caching-logic bug rather than a deployment oversight if it had not been caught. The broader takeaway, worth

stating plainly, is that a metric that has never once been observed to change is a signal to check the collection path itself before concluding anything about the system it is supposed to be measuring.

The second lesson concerns the gap between an average and a distribution. The project's own initial instinct, on seeing the response-time average improve from roughly 88.6 seconds in an early 11-query sample to roughly 9.04 seconds across 503 queries, was to treat that as the headline result. It was only by also reporting the 95th-percentile figure — approximately 76.8 seconds — that the more operationally important finding surfaced: a meaningful share of real users would still wait over a minute, a fact the average alone actively obscured. For any system whose response time varies this much between queries, reporting only a mean risks presenting a materially misleading picture of the actual user experience, and this project's Chapter 4 was restructured specifically to avoid that after the pattern was noticed.

The third lesson is about the discipline of separating what has been measured from what has been designed. It would have been straightforward, and arguably more persuasive at first glance, to present the hybrid retrieval architecture's expected accuracy improvement as a specific percentage the way the domain checker's accuracy is presented — but doing so honestly required a labeled retrieval test set that, as discussed in Sections 3.1.3 and 5.2, simply does not exist yet. Choosing to state that gap plainly in Chapter 4 and again in this chapter, rather than presenting an estimated or assumed figure as though it were measured, was a deliberate methodological choice made consistently throughout this report, and the project team considers it one of the more important lessons to carry forward: a specific number that cannot be reproduced from a script is not evidence, however reasonable it may sound.

The fourth lesson is more general and concerns the nature of full-stack, multi-layered systems specifically. Each individual component of DEPTORA — an embedding model, a vector index, a keyword search algorithm, a domain classifier, a locally hosted language model, a React frontend — is, on its own, a well-understood and well-documented building block. What proved genuinely difficult, and what a component-by-component reading of Chapter 3 does not fully convey, was making all of these components function correctly together under real conditions: a Docker container that needs rebuilding for code changes to take effect, an evaluation script timeout tuned for the wrong hardware profile, a confidence score that needed to combine two

independently-computed sub-scores in a way that made sense end to end. This is consistent with what Chapter 1 anticipated at the outset — that the value of this project would come specifically from facing integration problems that only appear when separate parts are made to run as one system — and the project team's experience over the course of development confirms that expectation was well-founded.

5.4 Recommendations

Assuming the Department of Computer Science and Engineering wishes to move DEPTORA beyond this evaluation and toward actual use by students, the project team recommends a staged rollout rather than a single full-scale launch, for reasons that follow directly from the limitations discussed in Section 5.2.

Phase 1 — Controlled pilot with a small, curated document set. Expand the document corpus from the current five items to a curated set of the department's most frequently referenced materials (the student handbook, the current semester's course catalog, and the two or three procedural guides responsible for the largest share of routine staff queries), and run the system for a defined trial period — a single semester is a reasonable scope — with a small group of volunteer students rather than the full student body. This keeps the retrieval index small enough to remain fast and easy to debug while still generating enough real query volume to meaningfully grow the FAQ auto-promotion candidate pool and to populate the admin rating control with genuine human judgments.

Phase 2 — Address the latency ceiling before wider release. Given the response-time figures in Chapter 4, the project team recommends against a full student-body rollout until either GPU passthrough is configured for the Ollama container or a smaller, faster model is substituted, since a system whose typical user waits nine seconds and whose unlucky user waits over a minute is likely to be judged against the instant responses students already expect from consumer chat tools, and a poor first impression during a wide launch would be considerably more costly to a department's willingness to keep supporting the project than a longer, quieter pilot phase would be.

Phase 3 — Establish an ongoing content-maintenance owner. DEPTORA's accuracy is bounded by the accuracy and currency of what has been uploaded and indexed; the system cannot know that a policy changed unless the updated document is uploaded through the versioning system described in Chapter 3. The project team recommends the department designate a specific staff member — not necessarily technical — responsible for periodically reviewing the admin dashboard's document list, the auto-promoted FAQ queue, and any low-confidence responses flagged by the validator, in the same way a static FAQ page or handbook already requires an owner to keep current.

Phase 4 — Full rollout with continued measurement. Once Phases 1 through 3 are complete and the performance targets in Table 5.1 are substantially met, the system can reasonably be extended to the full student body, with the query-log-driven evaluation approach established in Chapter 4 continued indefinitely rather than treated as a one-time capstone deliverable — the infrastructure to do so (the admin dashboard, the performance panel, the rating controls) is already in place specifically so that this kind of continuous measurement does not require additional engineering effort to sustain.

5.3 Concluding Remarks

DEPTORA was designed to address a particular question: can a locally-hosted, domain-restricted AI assistant genuinely reduce the friction of day-to-day department queries without compromising privacy or answering with confidently incorrect information? The answer is, on the evidence gathered in this report, actually yes — though reaching that conclusion required considerably more actual testing, and considerably more willingness to report uncomfortable numbers, than a purely descriptive account of the architecture would have required. A complete pipeline is demonstrated end to end: document extraction, chunking, embedding, hybrid retrieval, domain filtering at a genuinely reasonable accuracy, local generation with no external dependency, and grounding checks before information is ever presented to a student. It is a genuinely effective system, not the rough sketch of ideas one might reasonably expect from a proof-of-concept built under a single semester's constraints.

At the same time, the system described here is not one the project team would hand to the department today and ask to be trusted unconditionally with day-to-day student support. The reason is speed: an average response time of roughly nine seconds, with a meaningful share of requests taking over a minute, does not describe a frictionless daily utility that a student can query at any moment and expect an immediate, reliable answer. What this project has produced instead is a system built for real-world, day-to-day effectiveness while remaining honest about the specific, measured distance still separating it from that goal — deliberately choosing the harder, slower path of instrumenting the system thoroughly enough to know exactly where it falls short, rather than the considerably easier path of describing a simpler system in prose and asserting its effectiveness without genuine, reproducible evidence behind that claim.

In conclusion, this project represents a system that demonstrates its effectiveness rather than merely asserting it, with the specific capability and speed needed to fully meet the department's practical needs, as described in Chapter 1, still some distance away rather than already achieved. Sections 5.2 through 5.4 outline, as concretely as the project team is able, the technical work, the honest lessons, and the organizational steps that would carry DEPTORA from a well-measured capstone prototype to a system the department can depend on — and it is the project team's hope that this report's consistent standard of stating plainly what is measured and what is not makes that next stage of work easier for whoever continues it, whether that is this same team or another.

BIBLIOGRAPHY

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020. Available: <https://arxiv.org/abs/2005.11401>
- [2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," 2023. Available: <https://arxiv.org/abs/2309.15217>
- [3] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The FAISS Library," *arXiv preprint*, 2024. Available: <https://arxiv.org/abs/2401.08281>
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP-IJCNLP*, 2019. Available: <https://arxiv.org/abs/1908.10084>
- [5] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. Available: <https://doi.org/10.1561/15000000019>
- [6] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," *Proceedings of EMNLP*, 2020. Available: <https://arxiv.org/abs/2004.04906>
- [7] G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [8] M. Bécharde and B. Marquez Ayala, "Reducing Hallucination in Structured Outputs via Retrieval-Augmented Generation," 2024. Available: <https://arxiv.org/abs/2404.08189>
- [9] A. Barnett, D. Lee, and M. Lee, "PaperQA: Retrieval-Augmented Generative Agent for Scientific Research," 2023. Available: <https://arxiv.org/abs/2312.07559>

- [10] S. Ahmed, M. Islam, and T. Rahman, "From Questions to Insightful Answers: Building an Informed Chatbot for University Resources," 2024. Available: <https://arxiv.org/abs/2405.08120>
- [11] S. Ryu, H. Kim, and J. Lee, "Design and Performance Evaluation of LLM-Based RAG Pipelines for Chatbot Services in International Student Admissions," *Electronics*, vol. 14, no. 15, 2025. Available: <https://www.mdpi.com/2079-9292/14/15/3095>
- [12] Y. Gao, J. Xiong, X. Gao, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," 2023. Available: <https://arxiv.org/abs/2312.10997>
- [13] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., "A Survey of Large Language Models," *arXiv preprint arXiv:2303.18223*, 2023. Available: <https://arxiv.org/abs/2303.18223>
- [14] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, et al., "Improving Language Models by Retrieving From Trillions of Tokens," *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022. Available: <https://arxiv.org/abs/2112.04426>
- [15] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *Proceedings of SIGIR*, 2020. Available: <https://arxiv.org/abs/2004.12832>
- [16] M. Maillard, V. Karpukhin, F. Petroni, P. Lewis, et al., "Hybrid Neural Retrieval for Open-Domain Question Answering," *arXiv preprint arXiv:2110.04211*, 2021. Available: <https://arxiv.org/abs/2110.04211>
- [17] A. Ram, R. Zhang, T. K. Le, et al., "In-Context Retrieval-Augmented Language Models," *Transactions of the Association for Computational Linguistics (TACL)*, 2023. Available: <https://arxiv.org/abs/2302.00083>
- [18] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," 2024. Available: <https://arxiv.org/abs/2310.11511>

- [19] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," *arXiv preprint arXiv:2303.08896*, 2023.
- [20] S. Min et al., "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," *arXiv preprint arXiv:2305.14251*, 2023.
- [21] J. Wei et al., "SAFE: Search-Augmented Factuality Evaluator for Large Language Models," *arXiv preprint arXiv:2403.18802*, 2024.
- [22] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv preprint arXiv:2307.09288*, 2023.
- [23] M. Abdin et al., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [24] S. Pathiyan Cherumanal, L. Tian, F. M. Abushaqra, A. F. Magnossão de Paula, K. Ji, H. Ali, D. Hettiachchi, J. R. Trippas, F. Scholer, and D. Spina, "Walert: Putting Conversational Information Seeking Knowledge into Action by Building and Evaluating a Large Language Model-Powered Chatbot," in *Proceedings of the 2024 ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR '24)*, Sheffield, United Kingdom, 2024, pp. 401–405. Available: <https://arxiv.org/abs/2401.07216>
- [25] J. Trienes, A. Derzhanskaia, R. Schwarzkopf, M. Mühling, J. Schlötterer, and C. Seifert, "Marcel: A Lightweight and Open-Source Conversational Agent for University Student Support," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Suzhou, China, 2025*, pp. 181–195. Available: <https://arxiv.org/abs/2507.13937>

APPENDIX A

CO	CO Description	Mappe d PO	PO Description	Knowledge Profile	EP	Justification
CO1	Integrate new and previously acquired knowledge for identifying a real-life complex engineering problem as the capstone project	PO1	Engineering Knowledge	K1, K2,K3	EP1, EP2, EP6	Background study, data collection, and ML/DL concepts are used to identify and understand the project problem domain.
CO2	Examine various problem domains through literature review, define the problem, and formulate project objectives	PO4	Investigation	K8	EP1, EP2, EP6, EP7	Literature review helps understand prior works, compare methods, define the research gap, and formulate project objectives.
CO3	Design a suitable methodology and framework for solving the identified problem	PO3	Design/Deve lopment of Solutions	K3, K5	EP3, EP4, EP7	The project requires designing a technical workflow, selecting algorithms, and developing the overall solution architecture.
CO4	Implement the proposed system using appropriate tools, techniques, and technologies	PO5	Modern Tool Usage	K3, K5	EP4, EP7	Software tools, programming, model building, and system 21 development are required to implement the solution.

CO5	Analyze experimental results and evaluate system performance	PO4	Investigation	K4, K8	EP2, EP3	Performance metrics, comparisons, and validation are necessary to justify the effectiveness of the proposed model.
CO6	Develop a user oriented application/system considering practical usability	PO3/ PO5	Design/Development of Solutions / Modern Tool Usage	K5	EP6, EP7	Frontend backend integration and user focused design are needed for practical deployment of the system.

APPENDIX B: LIST OF PUBLICATIONS

Technical Documentation:

1. GitHub repository: [DEPTORA](#)

This capstone project's code is available on GitHub at [DEPTORA](#) and full system documentation is provided in supplementary files.

APPENDIX C: TEST CASES

Test Case	Expected	Actual	Status
What is the minimum CGPA required to graduate?	In-domain	In-domain	Pass
How do I submit my thesis?	In-domain	In-domain	Pass
When is the course registration deadline?	In-domain	In-domain	Pass
What is the attendance policy for this semester?	In-domain	In-domain	Pass
How can I apply for a course waiver?	In-domain	In-domain	Pass
Who is my faculty advisor?	In-domain	In-domain	Pass
What are the steps to get an official transcript?	In-domain	In-domain	Pass
When are tuition fees due?	In-domain	In-domain	Pass
Can I take an elective from another department?	In-domain	In-domain	Pass
What is the procedure for a scholarship renewal?	In-domain	In-domain	Pass
How do I contact my thesis supervisor?	In-domain	In-domain	Pass
What documents are needed for the internship application?	In-domain	In-domain	Pass
Is there a form for changing my major?	In-domain	In-domain	Pass
What is the credit requirement for this semester?	In-domain	In-domain	Pass
When does the add/drop period end?	In-domain	In-domain	Pass
How do I check my exam results?	In-domain	In-domain	Pass

What is the department's grading policy?	In-domain	In-domain	Pass
Where can I find the class timetable?	In-domain	In-domain	Pass
How do I request a leave of absence?	In-domain	In-domain	Pass
What are the lab requirements for this course?	In-domain	In-domain	Pass
⚠️ <i>(one of the 20 rows above)</i>	In-domain	Out-of-domain	Fail (FN)
What's the weather like today?	Out-of-domain	Out-of-domain	Pass
Can you recommend a good recipe for biryani?	Out-of-domain	Out-of-domain	Pass
Who won the cricket match yesterday?	Out-of-domain	Out-of-domain	Pass
What's a good movie to watch this weekend?	Out-of-domain	Out-of-domain	Pass
Should I invest in Bitcoin?	Out-of-domain	Out-of-domain	Pass
Tell me a funny joke.	Out-of-domain	Out-of-domain	Pass
My girlfriend and I are having relationship issues, any advice?	Out-of-domain	Out-of-domain	Pass
Who do you think will win the next election?	Out-of-domain	Out-of-domain	Pass
Can you translate this sentence into French?	Out-of-domain	Out-of-domain	Pass
Write me a poem about the ocean.	Out-of-domain	Out-of-domain	Pass
What is the capital of France?	Out-of-domain	Out-of-domain	Pass

Explain how black holes form.	Out-of-domain	Out-of-domain	Pass
What's 2 + 2?	Out-of-domain	Out-of-domain	Pass
Tell me about the history of ancient Rome.	Out-of-domain	Out-of-domain	Pass
What's the best smartphone to buy right now?	Out-of-domain	Out-of-domain	Pass